AI ENGINEER INTERVIEW PLAYBOOK

Underwriting Solution Jarvis AI Foundry

How the solution was built, how it evolved from ChromaDB to PostgreSQL to Azure AI Foundry, how the human-in-the-loop gates work, and 100+ questions with answers you can say out loud.

Built from the actual codebase: EmailIntentAgent, the MCP tool server, the Foundry extraction service, the risk scorer and the Guidewire handlers.

The UI is intentionally out of scope - you describe it as generated with Replit and keep the conversation on the agent, MCP, data and orchestration layers.

Claims solution gets its own playbook next.

Contents

1. How to use this playbook . . . 3

2. Roadmap . . . 4

3. Your pitch . . . 6

4. The story: how the solution evolved . . . 8

5. Architecture deep dive . . . 13

6. The 16 phases, as they run today . . . 18

7. Data extraction deep dive (Azure AI Foundry) . . . 22

8. Human-in-the-loop gates deep dive . . . 25

9. Risk scoring, perils and decisioning . . . 28

10. Guidewire integration (Phases 13 and 16) . . . 31

11. Reliability, observability, security, deployment . . . 32

12. Engineering stories (STAR) . . . 34

13. Honest gaps - and how to say them well . . . 37

14. Q&A bank . . . 39

15. Hands-on drills and cheat sheet . . . 53

1. How to use this playbook

The goal is not to memorise this document. The goal is that after one careful read and a few spoken run-throughs you can do four things in an interview: **tell the story** of the project in three minutes, **draw the architecture** on a whiteboard, **answer follow-up questions** without notes, and **defend your trade-offs** - including the ones you would change today.

Part	What you get	When to use it
2 Roadmap	14-day plan, 3-day compressed plan, what each interview round tests	Day 0, then check off daily
3 Pitch	30-second and 2-minute versions you can say verbatim	Learn first - this is your opening
4 The story	Chapter-by-chapter evolution with the real reasons and the Phoenix prompt-version timeline	Learn second - interviewers love 'how did it evolve'
5-11 Deep dives	Architecture, 16 phases, extraction, HITL gates, risk scoring, Guidewire, reliability	Day 2-7, whiteboard each one
12 Engineering stories	10 STAR stories with real numbers from the code	Behavioural + 'tell me about a hard problem'
13 Honest gaps	What the code does not do, and how to say it without sounding weak	Read twice - prevents over-claiming
14 Q&A bank	100+ questions with spoken-style answers	Read aloud; cover the answer and try first
15 Cheat sheet	Numbers, names, endpoints, tables, questions to ask them	Night before and morning of

Legend

SAY THIS
Phrasing you can use almost word for word.

WATCH OUT
A trap: something easy to over-claim, a likely follow-up, or a detail to double check against your memory.

IN THE CODE
A fact read directly from the repository. If you are challenged, this is solid ground.

YOUR REASONING - adjust to what you actually remember
The repository shows *what* was built but not always *why*. Where the reason is my inference, it is in this purple box. Replace it with what you actually remember - your real reason always beats mine.

Three ground rules

- **Do not name the client.** A prompt-version comment in the code references a specific customer template. In interviews say 'a commercial P&C insurer'. Also never recite hostnames, container names, keys or internal URLs.
- **Do not claim what the code does not do.** Part 13 lists the places where the implementation is simpler than its name suggests (sanctions screening, eligibility rows, checkpointing). Describe them accurately and they become strengths: you understand the gap and the fix.
- **Own your slice.** Decide in advance which parts you personally built, and say 'I' for those and 'we' for the rest. The pitch in Part 3 has a fill-in line for this.

2. Roadmap

Fourteen days is comfortable for two years of experience plus a project this size. Each day has one outcome you can test yourself on. If you only have three days, use the compressed plan underneath.

Day	Focus	You are done when you can...
1	Pitch and story (Parts 3-4)	Say the 2-minute pitch and the 7-chapter evolution without notes
2	Architecture and MCP (Part 5)	Draw agent - MCP - Azure services - Blob - UI from memory and explain each arrow
3	The 16 phases (Part 6)	Name each phase's tool, input, output; say where the three gates sit
4	Data extraction (Part 7)	Explain LOB match, per-document classify, chaining, ACORD routing, primary document
5	HITL gates (Part 8)	Explain halt-and-resume, the pipeline lock, and why not interrupt() in production
6	Risk, perils, Guidewire (Parts 9-10)	Compute a composite score by hand; list the five Guidewire calls
7	Reliability and security (Part 11)	List 8 failure modes with the mitigation in the code; give 5 security improvements
8	LangGraph theory + code drill	Write a minimal tool-calling graph; explain reducers, checkpointers, interrupt, Send
9	MCP + agents theory	Explain MCP primitives and transports; ReAct; supervisor vs swarm; tool-calling mechanics
10	RAG theory	Explain chunking, embeddings, HNSW vs IVF, hybrid search, reranking, RAG metrics
11	LLM evaluation, safety, cost	Design an eval harness for your extraction; explain prompt injection defences
12	System design drill	Design 'underwriting copilot at 10x volume' in 30 minutes on paper
13	Mock interview aloud	Run 40 questions from Part 14 out loud, record, review the weak answers
14	Gaps, cheat sheet, questions to ask	Re-read Part 13 and Part 15; sleep

If you only have three days

- **Day A (3 h):** Parts 3, 4 and 8. Pitch, story and gates. These three decide whether the project sounds real.
- **Day B (3 h):** Parts 5, 7 and the LangGraph and MCP questions in Part 14. Whiteboard the architecture and the extraction flow twice.
- **Day C (3 h):** Part 11, Part 13, then 30 random questions out loud. Finish with Part 15.

What each round actually tests

Round	What they are checking	Lead with
Recruiter / HR screen	Does the stack match? Can you communicate scope clearly?	30-second pitch, keyword-dense: LangGraph, MCP, Azure AI Foundry, RAG, HITL, Guidewire
Technical deep-dive on the project	Ownership, depth, whether trade-offs were yours	The evolution story, then extraction and gates
Hands-on / coding	Python, async, API design, prompt and tool design	Retry with backoff, thread-pool fan-out, SSE generator, minimal LangGraph (Part 15 drills)
System design	Scale, reliability, cost, trade-offs	Part 11 and the 10x scaling answer in Q&A
Hiring manager / behavioural	Judgement, failure handling, learning speed	STAR stories in Part 12; honest-gaps framing

How to practise

- Cover the answer, say yours aloud for 60 seconds, then compare. Writing answers down is much less effective than saying them.
- Redraw the four diagrams (architecture, gate flow, extraction, risk scoring) on paper until you can do each in under two minutes.
- Keep your own notes on LangChain, LangGraph, MCP and agents open next to Part 14 - every project answer here has a theory twin there.

3. Your pitch

30 seconds

SAY THIS
I work on an agentic AI platform that automates property underwriting for a P&C insurer. A broker emails a submission - an ACORD form, a statement-of-values spreadsheet, loss runs. A LangGraph agent, backed by an MCP tool server on Azure, indexes the attachments in an Azure AI Foundry vector store, classifies each document, extracts structured fields, screens the insured, scores the risk, and then - after an underwriter approves at three human-in-the-loop gates - produces a Guidewire quote, a quote PDF and binds the policy.

2 minutes

- **Problem.** Underwriters spend hours per submission reading messy emails and attachments, re-keying data into Guidewire, and running checks. The inputs are unstructured, the workflow has 16 steps across several systems, and the final decisions are regulated, so a human must stay in charge.
- **Solution.** Every capability is a FastAPI endpoint exposed as an MCP tool - 34 tools in about two dozen modules. A LangGraph agent loads a versioned prompt from Arize Phoenix, discovers the tools over streamable HTTP, and drives the phases.
- **Data extraction.** This is the part I would highlight. We started with ChromaDB, moved to PostgreSQL with pgvector, and now use Azure AI Foundry: one vector store per submission, every document classified and extracted independently, in parallel, with ACORD forms routed to a custom Document Intelligence model.
- **Governance.** Three gates - document review, eligibility, risk acceptance. The graph halts deterministically, and each gate has its own resume endpoint, so state lives in Blob and PostgreSQL rather than in agent memory.
- **Rigor.** The LLM reads, classifies, extracts and explains; deterministic code scores, maps and decides. We tuned latency with measured experiments - for example chaining two Responses API calls cut a 111-second classify-plus-extract to 30 seconds.
- **Integration.** Quotes and bind-and-issue go through Guidewire's REST APIs for residential and commercial property.

Say your ownership explicitly

WATCH OUT
Fill in the brackets with the truth. Example: 'I owned the [data extraction pipeline / agent and gate orchestration / MCP tool layer] and contributed to [x]. The React workboard was generated with Replit, so I will keep my answers on the backend.' Interviewers detect inflated ownership within two follow-ups.

Impact slots to fill with real numbers

Metric	Your number	Where to get it
Submissions processed per day / week	[ ]	Blob/Excel ingestion log, Workboard counts
Manual time per submission before vs after	[ ]	Ask the business analyst or underwriters
Fields auto-extracted correctly	[ ]	Spot-check 20 submissions against a gold sheet
Straight-through rate (no edits at Gate 1)	[ ]	Approval logs
Cost per submission (LLM tokens)	[ ]	Foundry usage + token_usage in extraction output

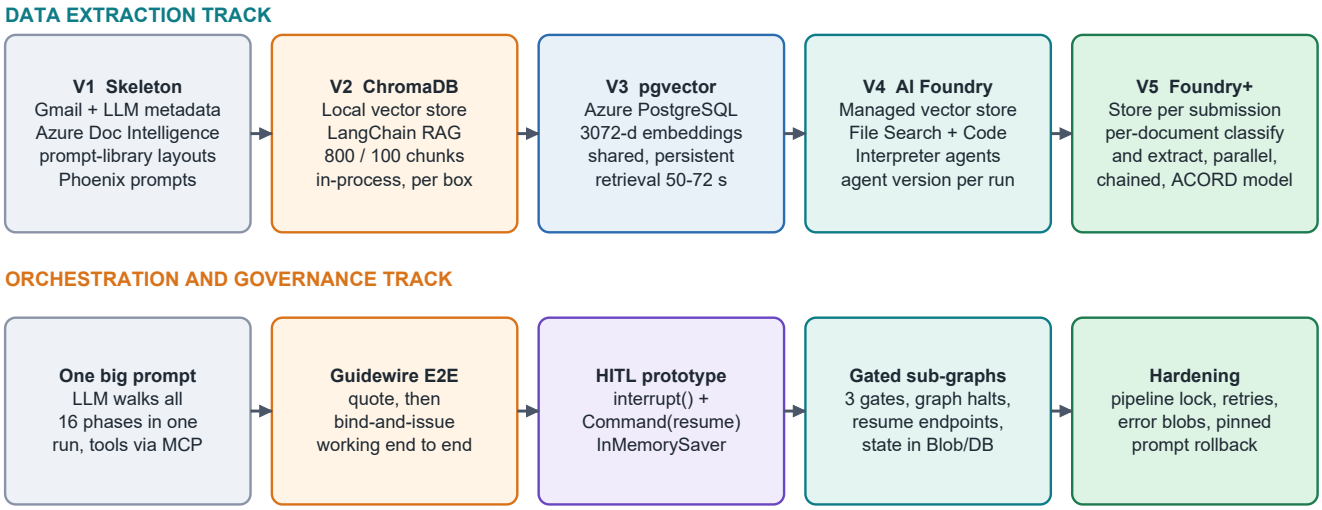
IN THE CODE
Numbers already in the code that you can cite as *engineering* results: a 56-of-56-field extraction on a full-size SOV in 28.8 s; classify-plus-extract 111 s → 30.5 s by chaining; a combined summary that used to take 150+ s now a cheap text call. Say 'in our testing', not 'in production'.

Three sentences to leave them with

- The LLM reads and explains; deterministic code scores, maps and decides.
- Human gates are resume points, not paused processes - the artifacts live in Blob and PostgreSQL, so a restart never loses an approval.
- Extraction moved from ChromaDB to pgvector to Foundry because each step removed a measured pain, not because of fashion.

4. The story: how the solution evolved

Interviewers rarely ask 'what is your architecture' - they ask 'how did you get here and what went wrong'. This chapter is that answer. Each chapter follows the same shape: **what we built → what hurt → what we changed**. The sequence comes from the code structure and from the Phoenix prompt-version comments pinned in `server.py`.



Order is relative. Phoenix prompt versions pinned in server.py (v30 -> v103) are the best evidence of sequence.
Chroma -> pgvector -> Foundry is the extraction history you described; the orchestration track is read from the code.

> **YOUR REASONING - adjust to what you actually remember**
> Order of the extraction track (Chroma → pgvector → Foundry) is what you told me. Order of the sub-stages inside Foundry, and the reasons given for each move, are inferred from file structure and code comments. Check each chapter against your memory before you present it.

Chapter 0 - The business problem

A broker sends a **submission**: a cover email plus attachments. For property insurance the attachments are typically an **ACORD** application form (a standardised, fixed-layout form), a **statement of values (SOV)** spreadsheet listing buildings and values, and **loss runs** (claims history). The underwriter must work out the line of business, read the key facts, check eligibility, assess risk, price it in Guidewire and respond to the broker. It is a strong agent use case: unstructured inputs, a multi-step workflow touching external systems, and regulated decisions that need human sign-off.

Chapter 1 - The skeleton: one agent, MCP tools, Blob as the bus

- **Built:** each capability as a FastAPI route, exposed to the agent through `fastapi_mcp`. A LangGraph agent with a single long system prompt, stored in Arize Phoenix, that lists the phases and the tool for each. Gmail API for ingestion; an LLM call (JSON mode, temperature 0.3) turns the email into property name, address, type, broker and customer.
- **Extraction foundations:** ACORD forms through an Azure Document Intelligence custom model; other documents through a prompt library in PostgreSQL (rows of LOB × document type × extraction rules).
- **Integration bus:** every artifact is written to Azure Blob Storage and passed to the next phase as a URL. Agents never call the UI.
- **What hurt:** the LLM sequenced all the phases from one prompt; the prompt grew with each phase; and any tool rename silently broke it (you can see this in a later prompt version that fixes a wrong tool name).

> **SAY THIS**
> We made every capability an MCP tool so the agent, tests and the chat feature all share one tool surface, and we kept prompts in Phoenix so we could iterate on behaviour without redeploying.

Chapter 2 - ChromaDB: prove the idea fast

- **Built:** PDFs → Azure Document Intelligence text → `RecursiveCharacterTextSplitter` (chunk 800, overlap 100) → Azure OpenAI embeddings → a local Chroma collection, with a retriever tool that answered field questions via an LLM chain.
- **Why it was a good first step:** zero infrastructure, runs on a laptop, lets you validate extraction prompts and layouts quickly. The corresponding agent prompt was the first pinned version (v30).
- **What hurt:** the store lives on the MCP server's local disk, which does not fit a containerised, redeployable service; no isolation between submissions; no SQL; every redeploy is a re-index question.

> **YOUR REASONING - adjust to what you actually remember**
> The code shows Chroma was local and in-process. 'Does not fit containers' is the natural reason, but if your real trigger was something else (a demo to the client, a team decision), use that.

Chapter 3 - PostgreSQL with pgvector: make it shared and durable

- **Built:** vectors moved into Azure PostgreSQL via the pgvector extension - a `pgvector_store` table, 3072-dimensional embeddings, `register_vector`, a retrieval chain with a prompt from Phoenix (pinned v34).
- **Why:** one durable, shared store in the same database family we already used for the prompt library; SQL filtering; survives restarts and redeploys.
- **One index, two consumers:** the description attached to an earlier Phoenix prompt version (the exact version number was not given, so I place it in this era) says it dropped the separate pdf_vectorizer step and used `query_pdf_vectorizer`, which stores combined chunks that serve both the **chat** feature and **non-ACORD data extraction through semantic search**. ACORD forms still went to the custom Document Intelligence model.
- **What hurt (from the code):** the retrieval tool 'regularly takes 50-72 s', which is why the MCP client's read timeout is 600 s. We owned chunking, embedding, indexing, retrieval and the answer prompt. Spreadsheets chunk badly. Every field extraction became a retrieval-quality gamble.

> **SAY THIS**
> In the pgvector era one set of chunks did two jobs, chat and semantic extraction for non-ACORD documents. Foundry kept the 'one index per submission, many consumers' idea, but extraction stopped depending on top-k retrieval.

> **IN THE CODE**
> In `server.py` the MCP client config carries the comment: 'pgvector_retrieval_mcp regularly takes 50-72s, so 600s gives ~10x headroom'. That one comment is your evidence for why pgvector was painful.

Chapter 4 - Layout intelligence and the first full end-to-end run

- **Layout prompt learned to read text as well as PDF** (v65), because some submissions arrive only as an email body.
- **A production prompt** for a specific client's document template (v68).
- **Regex normalisation of layout type** (v80), because the LLM's label wobbled in spelling. First run through the draft-email phase with Guidewire, but without bind-and-issue.
- **Bind-and-issue added** (v82): first complete end-to-end run including Guidewire.
- **Contract fixes:** a prompt that named the wrong retrieval tool (v83); an extraction phase that used the raw layout type instead of the matched document type (v84).

> **SAY THIS**
> The lesson from this phase: an LLM-driven pipeline needs *contracts* between prompt and tools. We started normalising outputs in code and pinning prompt versions.

Chapter 5 - Azure AI Foundry: stop owning the retrieval stack

This is the chapter to be most precise about, because it is where the design is best.

5a. Foundry as the extraction backend (pinned prompt v97)

- Replace our own chunk/embed/index/retrieve with Foundry's managed **vector stores** and **File Search**, plus **Code Interpreter** for spreadsheets (an SOV is a table, not prose).
- A *runtime agent version* was created per run (`PromptAgentDefinition` with File Search and Code Interpreter), used for extraction, then deleted in a `finally` block along with its context blob.

5b. Step-wise Foundry tools

- Small tools: `index_submission`, `detect_lob_mcp`, `detect_layout_type_mcp`, `extract_fields_mcp`.
- **Discovery:** Foundry's file-search attributes and filters did not persist on this deployment, so a shared store cannot be scoped to a submission. Decision: **one vector store per submission** so documents can never leak across submissions.
- **Incremental indexing:** a PostgreSQL table remembers which files are already uploaded; a growing email thread only indexes its new attachments.
- **What hurt:** layout detection was done at *submission* level, but one submission contains several documents of *different* types.

5c. One deterministic tool: process_submission_mcp (pinned prompt v99)

- Phases 4-6 (LOB detection, layout detection, extraction) collapsed into **index_submission + process_submission_mcp**. Fewer LLM-chosen steps means fewer places for the agent to go wrong and less token spend. (In the current v103 text, Phase 4 is the index step, Phase 5 is process_submission_mcp, and Phase 6 is an ACORD field-merge step.)
- **LOB at submission level, layout at document level.** Each document is attached directly (not searched), so one document can never contaminate another's classification.
- ACORD forms are detected by content and sent to the custom Document Intelligence model; everything else uses a chained classify-then-extract.
- Parallel across files (capped at 4), retry with exponential backoff on rate limits, a broker-supplied field list can override the configured fields, and the output names a **primary document** and a canonical `detected_lob` for downstream phases.

> **WATCH OUT**
> Do not say 'Foundry replaced RAG'. Say: 'We still use File Search for submission-wide questions - LOB detection and the chat feature - but for classification and extraction we attach the single document directly, because top-k chunk retrieval is the wrong tool for reading a whole form.'

Chapter 6 - Governance: from an interrupt prototype to gated sub-graphs

- **Prototype** (`server_hitl.py`): a `human_assistance` tool that calls LangGraph's `interrupt()`, an in-memory checkpointer keyed by `thread_id`, and a `/continue_chat` endpoint that resumes with `Command(resume=...)`. Here *the LLM decides when to ask a human*.
- **Production:** three fixed gates. After Phase 7 the graph halts because *code* detects that the summary tool succeeded. Each approval triggers a separate endpoint that starts a fresh narrow sub-graph for the next block of phases. No agent memory is needed across a gate - the inputs are URLs in the first message.
- **Full-pipeline mode:** a no-gate endpoint runs Phases 9-16 back to back, protected by a PostgreSQL lock so a double-click cannot start two runs.

> **YOUR REASONING - adjust to what you actually remember**
> Reasons you can give: (1) gates are policy, so they should be deterministic rather than an LLM judgement; (2) an in-memory checkpointer is lost when a container restarts, while a human may take hours to approve; (3) the UI already owns the approval state. If you remember a specific incident that triggered the change, lead with it.

Chapter 7 - Hardening and performance

- **Rate limits:** exponential backoff (2, 4, 8, 16, 32 s); concurrency capped at 4 after 6 concurrent gpt-5 calls triggered 429s.
- **Latency:** chain classify+extract with `previous_response_id`; use gpt-4o for both calls; summarise from already-extracted JSON instead of re-reading files.
- **Determinism moved into code:** the mapping from LOB code to the canonical 'RESIDENTIAL PROPERTY' or 'COMMERCIAL PROPERTY' string used to be left to the agent; it translated 'home' correctly once and passed 'commercial' through untranslated the next time, so the mapping now lives in `process_submission_mcp` (pinned v103).
- **Observability:** per-execution logs streamed to an append blob; a structured error record with the last tool invoked, written to its own container; SSE keep-alive comments every 20 s.
- **Prompt safety:** a prompt version that stored content as a plain string instead of a list of text blocks crashed prompt loading (v101); v102 fixed the shape. Pinned versions make this a one-line rollback.

Chapter 8 - Where it stands and what comes next

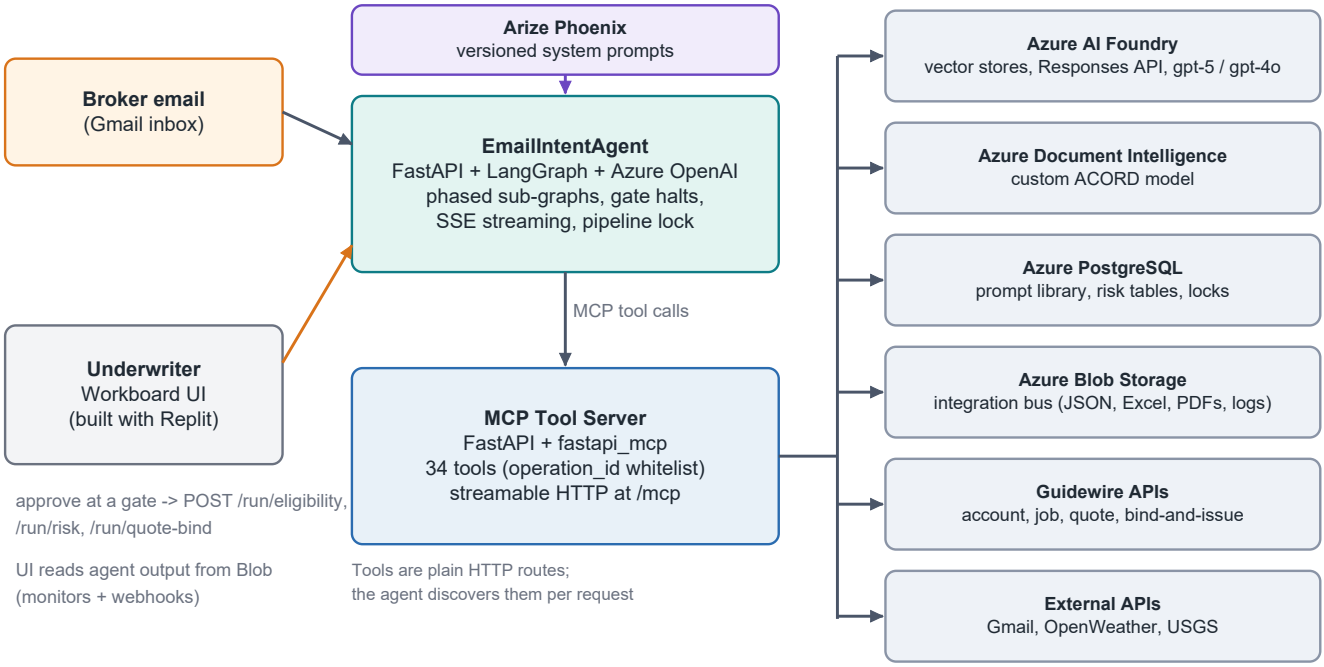
Today: 16 phases, three gates, five LOB codes classified (home and commercial wired through Guidewire), all extraction on Foundry. Natural next steps are in Part 13: a checkpointer-backed explicit graph, an evaluation harness, fuzzy sanctions matching and managed-identity auth.

Prompt-version timeline (your best evidence of sequence)

Pinned version	What changed (from the code comments)	What it teaches
v26 / v27	System and user prompts for email analysis (property, address, type, broker, customer)	Prompts also live inside tools, not only in the agent
v30	Agent prompt of the ChromaDB era	First pinned production flow
v34	Agent prompt for the pgvector retrieval tool	Storage change forced prompt change
v38	Worked up to the draft-email phase; layout prompt accepted PDF only	Incremental milestones
v65	Layout prompt accepts text as well as PDF	Real submissions are messy
v68	Production prompt for a client's template	Client-specific tuning
v80	Regex added to normalise layout type; end-to-end through draft, with Guidewire, no bind	Normalise LLM output in code
v82	Bind-and-issue added: Guidewire end to end	First complete run
v83	Fixed a prompt that named the wrong tool	Prompt-tool contract drift
v84	Extraction phase uses matched document type, not raw layout type	Chain outputs carefully
v97	Azure AI Foundry data-extraction phase	Foundry arrives
v99	Phases 4-6 replaced by index_submission + process_submission_mcp	Collapse LLM steps into one deterministic tool
v100	Phases 8-14 repointed to LOB and primary-document outputs; Phase 8 calls no tool	Dependent phases follow data model
v101	Broken: content was a string, not text blocks; load_prompt crashed	Pin versions; schema-check prompts
v102	Same text with the correct shape; Phase 7 → summarize_attachments_mcp	Fast rollback
v103 (current)	Canonical detected_lob passed to Phases 10, 11, 13; Phase 10 ends gracefully if null. Full text also has Phase 2 write_email_to_excel and a Phase 6 ACORD merge	Moved determinism out of the prompt; prompt text can drift from the tool registry

WATCH OUT
The IDs in the code are base64 strings of the form 'PromptVersion:N'. The numbers above are decoded from them. They show *relative* order; do not quote calendar dates for them.

5. Architecture deep dive



Arrows = runtime calls. Blob is the hand-off point between the agents and the UI.

5.1 Components and how you run them

Component	Entry point	Tech	Default port in code
MCP tool server	Jarvis/MCP/test_main.py	FastAPI + fastapi_mcp; sub-app mounted at /api/v1/email_intent_agent	5074
EmailIntentAgent	Jarvis/EmailIntentAgent/server.py	FastAPI + LangGraph + AzureChatOpenAI; SSE	5084
HITL prototype	Jarvis/EmailIntentAgent/server_hitl.py	interrupt() + InMemorySaver	8021
Standalone Eligibility / Risk agents	EligibilityAgent/, RiskAssessmentAgent/	Early per-stage agents with a human_assistance tool; superseded by gated sub-graphs	n/a
Underwriter Workboard (UI)	app_uw_v11	Express + React, built with Replit	2015

> **IN THE CODE**
> Ports above are the `uvicorn.run` defaults in each file. Deployed containers override them through environment variables and an internal DNS name, so say 'configurable' if asked.

How you start it (root folder is literally named `AI_Foundy_UW`; say 'AI Foundry UW' aloud):

```
# 1) MCP tool server - start first, the agent fetches tools from it
cd AI_Foundy_UW/Jarvis/MCP
source mcp_venv/bin/activate
python3 test_main.py

# 2) EmailIntentAgent
cd AI_Foundy_UW/Jarvis
source agent_venv/bin/activate
cd EmailIntentAgent
python3 server.py

# 3) UI
cd AI_Foundy_UW/app_uw_v11
npm run dev
```

- **Two virtual environments on purpose:** the agent needs LangGraph and the MCP client; the tool server needs the Azure SDKs, psycopg2, openpyxl and so on. Mixing them causes dependency conflicts and lets agent code import tool internals it should only call over HTTP.
- **Start order matters:** the agent calls the MCP server to discover tools on every request, so with the MCP server down the agent starts fine but each request fails fast at tool discovery.

5.2 Why MCP instead of plain LangChain tools

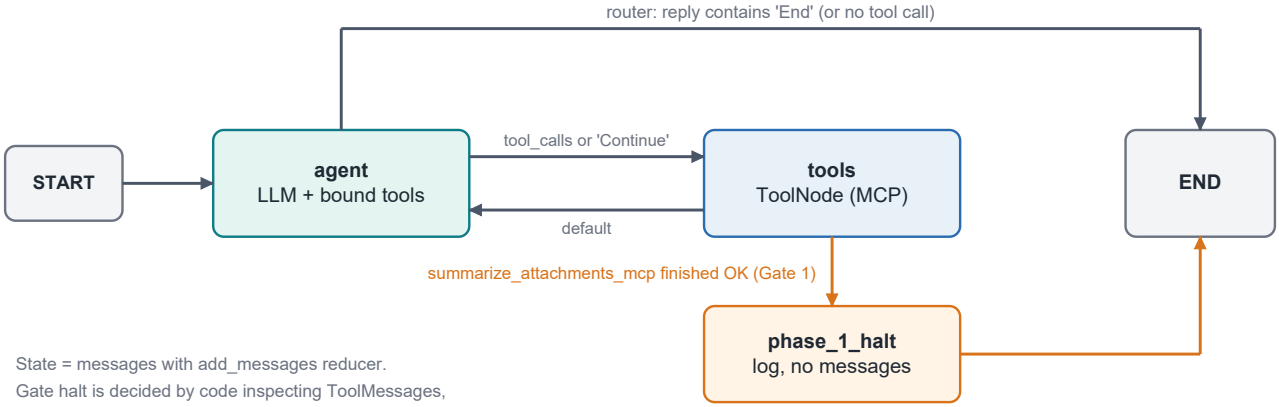
- **One tool surface, many clients.** The same tools are called by the LangGraph agent, by the Workboard's chat feature (which calls the chat-with-submission tool) and by tests and curl. Plain in-process LangChain tools would tie them to one Python process.
- **Independent deployment.** Tools change often (new LOB, new document type); the agent rarely does. The MCP server redeploys without touching the agent.
- **Language and runtime isolation.** Heavy dependencies (Azure SDKs, Excel, PDF) stay in the tool server.
- **Discovery at request time.** The agent asks the server for its tools on each call, so new tools show up without redeploying the agent. The price is one extra round trip per request.
- **Trade-off to admit:** a network hop per tool call, and a larger tool list in the model's context (see Part 13).

5.3 How tools are exposed (fastapi_mcp)

```
email_intent_agent_app = create_sub_app(title="email_intent_agent_mcps", ...)
email_intent_agent_app.include_router(email_router)        # all routes live in test_email_router.py
FastApiMCP(email_intent_agent_app,
           include_operations=["email_reader_mcp", ..., "process_submission_mcp",
                               "summarize_attachments_mcp"]   # 34 operation_ids
           ).mount_http()                                      # streamable HTTP at .../mcp
app.mount("/api/v1/email_intent_agent", email_intent_agent_app)
```

- Each FastAPI route's **operation_id** becomes the MCP tool name; its parameters become the JSON schema the LLM sees. Good summaries and docstrings are effectively prompt text.
- **include_operations is a whitelist.** A route with no matching entry is silently not a tool, and a typo does not raise an error. Whenever a tool 'does not exist' for the agent, check this list first.
- The same app is also plain REST at the parent path, which the agent uses for the pipeline lock (a deterministic call that should not go through the LLM).
- Legacy tools from earlier eras (Chroma and pgvector retrievers, old layout and extract tools) are still registered. The prompt, not the registry, decides which ones are used - Phase 8 of v103 even says 'do not call risk_type_checker_mcp, it is superseded'. A point to raise yourself under 'what would you clean up'.

5.4 The LangGraph agent




```
def create_custom_graph(model, tools, prompt, checkpointer=None, enable_phase1_halt=True):   # abridged
    graph_builder = StateGraph(State)                # State = {"messages": Annotated[list, add_messages]}
    llm_with_tools = model.bind_tools(tools)

    async def agent_node(state):
        msgs = [SystemMessage(content=prompt)] + state["messages"]
        return {"messages": [await llm_with_tools.ainvoke(msgs)]}

    def after_tools_router(state):
        if enable_phase1_halt and _find_tool_result_in_batch(
                state["messages"], "summarize_attachments_mcp") is not None:
            return "phase_1_halt"                    # Gate 1: code decides, not the LLM
        return "agent"

    graph_builder.add_node("agent", agent_node)
    graph_builder.add_node("tools", ToolNode(tools=tools))
    graph_builder.add_node("phase_1_halt", phase_1_halt_node)
    graph_builder.add_edge(START, "agent")
    graph_builder.add_conditional_edges("agent", router, {"tools": "tools", "End": END})
    graph_builder.add_conditional_edges("tools", after_tools_router,
                                        {"agent": "agent", "phase_1_halt": "phase_1_halt"})
    graph_builder.add_edge("phase_1_halt", END)
```

- **State** is just a message list with the `add_messages` reducer (append, update by id). All context for later phases travels as text - mostly Blob URLs - inside those messages.
- **Router protocol:** if the last AI message has tool calls, go to the tool node. Otherwise the prompt tells the model to answer exactly 'Continue' (loop back) or 'End' (stop), and the router maps those words to edges. Cheap and readable, but string matching on LLM output is brittle (Part 13).
- **Gate detection:** `_find_tool_result_in_batch` walks backwards through the trailing ToolMessages after the last AI message with tool calls, finds the call named `summarize_attachments_mcp`, matches the result by `tool_call_id`, rejects errors, and parses the JSON from the MCP content blocks. If it succeeded, route to the halt node. Matching by id avoids false positives from earlier turns and from parallel tool calls.
- **Bound tools:** `model.bind_tools(tools)` converts the 34 MCP tools into the provider's function-calling schema.

> **SAY THIS**
> The agent is a standard tool-calling loop, but the parts that matter for governance are not left to the model: the gate halt is a router that inspects tool results, and each gate has its own entry point.

5.5 One graph per gate segment

Endpoint	Runs	System prompt	recursion_limit	Notes
POST /chat	Phases 1-7 then halts at Gate 1	Phoenix, pinned v103	250	SSE stream, per-execution log, keep-alive, error blob
POST /run/eligibility	Phase 9 only	_PHASE_9_PROMPT (in code)	20	Gate 1 approval
POST /run/risk	Phases 10-12	_PHASE_10_12_PROMPT	75	Gate 2 approval; passes detected_lob
POST /run/quote-bind	Phases 13-16	_PHASE_13_16_PROMPT	75	Gate 3 approval
POST /run/full-pipeline	Phases 9 → 10-12 → 13-16	the three above	50 / 75 / 75	No gates; Postgres lock; 409 on duplicate
GET /health	-	-	-	Liveness

- **Narrow prompts per segment** include explicit GUARDRAILS ('call only X, Y and Z', 'respond with exactly End'). Smaller instruction surface means fewer wrong tool calls.
- **Low recursion limits** are a cheap circuit-breaker: Phase 9 is a single tool call, so 20 is plenty; a runaway loop fails quickly instead of burning tokens.
- **Context in the first message:** each endpoint builds a user message that contains the reference id, the extraction JSON URL, the canonical LOB and the LOB-detection blob URL. That message *is* the state hand-off across a gate.

5.6 Streaming, logging and errors

- **SSE streaming:** the graph runs in a background task that pushes events into an `asyncio.Queue`; the response generator waits up to 20 s for an event and otherwise emits an SSE comment (': keepalive') so proxies and load balancers never see minutes of silence.
- **Graph execution log:** a custom logging handler appends every step, node visit, tool call and tool result to an Azure **append blob** in near real time (IST timestamps), plus a summary of node counts and duration.
- **Error record:** any exception becomes an AGENT_ERROR JSON - exception class, message, elapsed seconds, seconds since last event, **last tool invoked**, traceback - written to its own container and streamed to the client as the final event.
- **Timeouts:** MCP client connect 120 s; SSE read 600 s, sized to 'slowest tool plus 10x headroom'.

> **WATCH OUT**
> The execution log writes full message content (tool outputs include insured names and addresses) to Blob. In a security question, volunteer that you would mask PII and set a retention policy.

5.7 Prompts and models

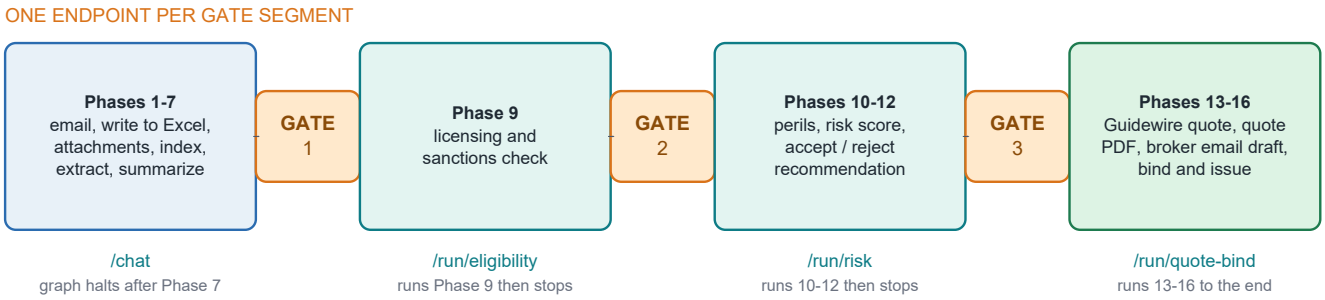
Where	Model / client	Prompt source	Note
Agent orchestration	AzureChatOpenAI (deployment from env)	Phoenix, pinned version id	Fetched on every /chat request
Email metadata extraction (tool)	AzureOpenAI chat, JSON mode, temp 0.3	Phoenix v26 (system) + v27 (user)	Property, address, type, broker, customer
LOB match, field spec detection, summary, chat	Foundry Responses API, gpt-5	In code (instructions strings)	File Search on the submission store
Document classify + extract	Foundry Responses API, gpt-4o, chained	In code, taxonomy from Postgres	Faster; same model both calls
ACORD extraction	Azure Document Intelligence custom model	n/a	No LLM generation
Risk justification	AzureOpenAI chat, temp 1, max 900 tokens	Phoenix v32	Explains; never decides

5.8 Data stores

Store	Holds	Used by
PostgreSQL - prompt/attribute DB	ref_prompt_library_final_layout (lob, document_type, extraction_rules JSON); submission_index_files (reference_id, vector_store_id, file_id, file_name, indexed_at)	Foundry extraction service
PostgreSQL - risk tables	brt01 ... brt08b lookup tables (many with int4range columns), risk_weightage, risk_score_band	Risk score calculator
PostgreSQL - peril data	PERIL_RISK_DATA keyed by latitude/longitude	Peril capture
PostgreSQL - workflow	submissions.full_pipeline_triggered_at; guidewire_data (submission_id, job_number)	Pipeline lock; Guidewire quote
Azure Blob Storage	Attachments, per-document extraction JSON, LOB JSON, peril and risk outputs, quote PDFs, Excel hand-off files, agent-logs, agent-errors	Everything; the integration bus
Azure AI Foundry	One vector store per submission; uploaded files	Extraction, summary, chat

6. The 16 phases, as they run today

The sequence below is taken from the **Phoenix v103 prompt** (the full text you pasted), cross-checked against the tool registry and the code. The numbering survives from the original flow, and two things changed along the way: the old separate LOB, layout and extraction steps collapsed into two tools (**Phases 4-5**), and **Phase 8 no longer calls any tool** because the LOB is already known from Phase 5. Section 6.1 explains how the prompt itself is built, including the places where its text and the code disagree.



Underwriter approves in the UI -> UI calls the next endpoint -> a fresh sub-graph starts with URLs in its first message.
No in-memory graph state survives a gate: the artifacts live in Azure Blob and PostgreSQL.

#	Phase	Tool (operation_id)	What it does	Output
1	Email read	email_reader_mcp	Polls Gmail; LLM extracts property name, address, type, broker, customer; mail body saved to Blob as body_{ref}.txt. The agent also judges whether the mail is insurance-related: **End** if not	reference_id, has_attachments, metadata
2	Write to Excel	write_email_to_excel	Appends one row for the email to the workbook named by EXCEL_FILENAME (default Fetched_emails_data_new.xlsx) in Blob. The Workboard's Excel monitor creates the submission from that file	excel_file, rows_count
3	Attachment download	email_attachment_mcp	Runs only if has_attachments is true, otherwise skipped. Saves each attachment to Blob as {ref}_attachment_...	Blob URLs
4	Index	index_submission	Creates or reuses the submission's vector store; uploads only new files; records them in Postgres. INDEX_CREATED, INDEX_UPDATED, ALREADY_INDEXED continue; an error or no files = **End**	vector_store_id
5	Process submission	process_submission_mcp	LOB match; per-document template detect, classify, extract; ACORD to custom model. is_insurance_submission false = **End**	primary_document, detected_lob, lob_output_file
6	ACORD field merge	acord_field_merger_mcp	Only when the primary document is ACORD: merge fields across the submission's documents. Present in the deployed MCP build (the local repo copy is behind and does not include it)	merged_primary_document_url
7	Summarise	summarize_attachments_mcp	One combined summary of the attachments (mail body excluded), built from already-extracted JSON; null for body-only mails; pushes the result to the Workboard	summary → **GATE 1**
8	(no tool)	-	LOB is known from Phase 5. An empty lob = **End**	-

#	Phase	Tool (operation_id)	What it does	Output
9	Licensing and sanctions	Licensing_and_Sanction_Checker_MCP	Finds insured name in the extraction JSON, compares with a sanctions list, writes an eligibility row to an Excel workbook in Blob. Full prompt: sanctioned = End. Segment prompt after Gate 1 ends either way	is_Sanctioned → **GATE 2**
10	Perils	peril_data_capture_mcp	Geocodes the address; earthquake count; nearest-neighbour peril attributes. detected_lob null (LOB not wired to quoting) = **End**	peril JSON (Blob URL)
11	Risk score	risk_score_calculator_mcp	24 parameters → 6 domains → composite 0-1000; band lookup; LLM justification	composite_score, band, justification
12	Accept / reject	accept_reject_risk_functionality_mcp	Maps composite to a decision band and an is_hitl_required flag	decision → **GATE 3**
13	Guidewire quote	guidewire_quote_mcp	Account, submission job, location, coverages, quote. Missing jobId or output URL = **End**	jobId, quote JSON URL
14	Quote PDF	Quote_Generator_MCP	Fills a template from extraction + quote data + LOB JSON (three Blob URLs)	PDF URL
15	Broker email	email_draft_mcp	LLM writes the body; a Gmail draft is created with the PDF attached. (The v103 text says 'create_draft', see 6.1)	draft
16	Bind and issue	guidewire_bind_and_issue_mcp	POST bind-and-issue for the job; saves response. Ends the run whether status is true or false	policy response URL

IN THE CODE
Tool names are the `operation_id` values in `test_email_router.py`; the sequence and branch rules come from the Phoenix v103 prompt, and Phases 9-16 also exist as the segment prompts in `server.py`. The pipeline lock (`acquire_pipeline_lock_mcp`) is a route that is deliberately *not* in the MCP whitelist - the agent reaches it over plain REST, so the LLM can never skip or fake it. That is also why the count is 34 tools, not 35 (counts are for the repo copy; the deployed build also has the ACORD field merger, so say 'about three dozen').

6.1 Anatomy of the Phoenix prompt (v103)

Expect 'walk me through your agent prompt' in every agentic interview. This is the shape of the real one, so you can describe it without notes.

Section	What it says	Why it exists
Role line	'An intelligent agent within the Insurance Underwriting Process', and at the end, 'the third layer of automation'	Anchors the model's job and frames the agent as one layer of a larger automation stack
Response protocol	Where a phase ends with a verdict, answer with exactly 'Continue' or 'End', with no punctuation or other text; 'if you output anything else, the system will fail'	The router maps those two words to graph edges, so the prompt is part of the control flow
16 numbered phases	Each names its tool, where every argument comes from (mostly 'from Phase 5'), and the rule for Continue versus End	The prompt is the wiring diagram between tools; state travels in the message history as Blob URLs
GUARDRAILS (6 rules)	Follow the sequence strictly; never skip a phase, call out of order or infer a missing output; if anything is missing, unclear, circular or you are unsure, answer End; only use the allowed strings	Fail closed: when in doubt, stop, because a wrong guess later means a wrong quote
Final instruction	'We do not have human in the loop in this flow, so ignore the human in the loop part'	See below: the gates are enforced in code, not in this text

Design ideas worth naming

- **Fail-closed control flow.** Every doubt resolves to End. The cost is that a confusing situation ends the run silently, so the execution log and the AGENT_ERROR record are what you debug from.
- **Two LOB values on purpose.** Phase 5 returns *lob* (raw taxonomy code such as 'home') for internal use, and *detected_lob* (the canonical 'RESIDENTIAL PROPERTY' or 'COMMERCIAL PROPERTY') that Phases 10, 11 and 13 must receive verbatim. A null detected_lob means the LOB is not wired to Guidewire yet, and Phase 10 ends gracefully.
- **The prompt describes the whole flow, but the code enforces the gate.** The main graph is built with the Gate 1 halt switched on, so it stops after Phase 7 whatever the prompt says. Phases 8-16 of this prompt are then re-run from the segment prompts in code, one block per approval. Treat the master text for Phases 8-16 as the original end-to-end definition (it is still what a run without the halt would follow).
- **Master prompt versus segment prompts.** The segment prompts are narrower copies with the details fixed: correct phase cross-references, the exact tool name for the draft step, and a single ending word per segment. Same sequence, tighter contract.
- **Tools deprecated inside the prompt.** Phase 8 says 'do not call risk_type_checker_mcp, it is superseded'. That is a clean example of using the prompt as the allow-list while old tools stay registered.

Where the prompt text and the code disagree

Item	What the prompt says	What the code has	How to say it
Phase 15 tool	'create_draft'	The operation_id is email_draft_mcp (create_draft is an internal function). The segment prompt uses the right name	'The model maps to the nearest tool; I would validate names in CI.'
Phase 9 tool	Licensing_&_Sanction_Checker_MCP	The operation_id is Licensing_and_Sanction_Checker_MCP; an ampersand is not valid in a function name	Same contract-drift point
Phase refs	Phase 13 keeps values 'for use in Phase 15'; Phase 16 reads values 'from Phase 12'	They are really Phase 16 and Phase 13; the segment prompt has them right	Stale after renumbering
Duplicate check	Not called anywhere in the prompt	attachment_checker_mcp is registered, but v103 never uses it	Dedup today is: email_reader skips repeated message ids within a call, and Phase 4 is idempotent
Phase 7 source	'sourced from the Foundry vector store'	Docstring in service.py: built from each attachment's already-extracted JSON, falling back to raw files only if no output exists	Trust the code; say 'text-only summary from extraction output'

WATCH OUT
Do not claim 'we validate that every tool in the prompt exists'. Today nothing does, and the gap above is the evidence. Offer the fix instead: at startup, extract tool names from the prompt and compare them with the list returned by get_tool_list, failing the deploy on a mismatch.

SAY THIS
The master prompt in Phoenix defines the full sixteen-phase contract and the fail-closed rules. The human gates are not in the prompt: the graph halts in code after the summary, and each approval starts a narrower prompt for the next block.

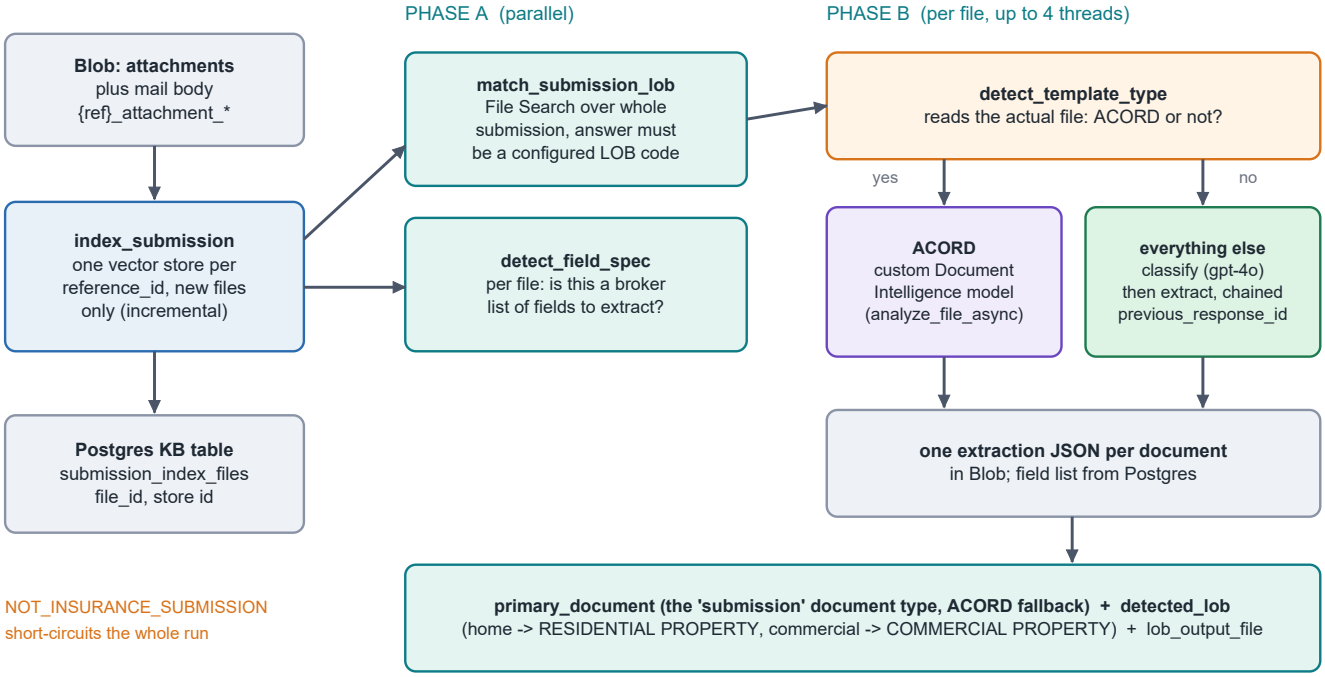
Why the pipeline is shaped like this

- **Cheap exits first.** Phase 1 stops mail that is not insurance-related, Phase 4 is idempotent so a re-run does not re-index, and Phase 5 short-circuits with NOT_INSURANCE_SUBMISSION so spam and marketing never reach the expensive steps.
- **Human review before expensive or irreversible work.** Gate 1 sits before checks and scoring; Gate 3 sits before anything is written to Guidewire.

- **Each phase writes an artifact and passes a URL.** That makes phases re-runnable in isolation and debuggable from Blob alone.
- **LOB drives behaviour.** Home maps to RESIDENTIAL PROPERTY, commercial maps to COMMERCIAL PROPERTY; other LOB codes (management liability, new submission, political) are classified and extracted but not yet wired to quoting, and Phase 10 ends gracefully for them.

7. Data extraction deep dive (Azure AI Foundry)

This is the technical centre of the project and where most questions will land. The code is in `MCP/AI_foundry_extraction_mcp/service.py` (handlers in `handler.py`).



7.1 Indexing: one vector store per submission

- **Why per submission:** file-search attributes/filters did not persist on this Foundry deployment (they came back *None* after upload and there was no update endpoint), so one shared store could not be scoped by reference id. A dedicated store makes cross-submission leakage impossible by construction.
- **Incremental indexing:** a PostgreSQL table (`submission_index_files`) records each uploaded file. Next time, the code diffs Blob file names against that table and uploads only the new ones - a broker replying with an extra attachment does not trigger a re-index.
- **Upload concurrency:** `upload_and_poll` blocks until Foundry finishes chunking and embedding, so uploads run in a thread pool (max 4).

7.2 Phase A: LOB and field-spec detection, in parallel

- **match_submission_lob:** one File Search call over the whole submission. The valid LOB codes are read from Postgres (*home, commercial, management_liability, newsub, political*) and the model must reply with exactly one of them or the marker `NOT_INSURANCE_SUBMISSION`. The result is validated; anything else raises.
- **detect_field_spec per file:** some brokers attach a list of the fields they want extracted. A direct-attach call asks 'is this a field specification?' and returns the field names as JSON, or `NOT_A_FIELD_SPEC`.
- These two touch different inputs and need nothing from each other, so they run concurrently. Trade-off noted in the code: if the submission turns out not to be insurance, the spec calls were wasted - acceptable because that case is rare.

7.3 Phase B: per-document, in parallel

- **Template detection:** an LLM reads the actual file and answers 'ACORD', 'MRC_SLIP' or NONE. Only ACORD is supported by the downstream path today; other templates fall back to the LLM path with a warning (so a new template never crashes the submission).

- **ACORD route:** the file's Blob URL goes to `analyze_file_async(blob_url, 'ACORD')`, which uses the Azure Document Intelligence custom model. Fixed-layout forms are what that model is good at; field positions are stable and confidence scores come with the output.
- **Everything else (loss runs, SOVs, cover letters, email bodies):** chained classify-then-extract.

Chained classify then extract

```
# call 1: attach the file, ask for the document_type (must be one of the LOB's configured types)
classify_response = responses.create(model="gpt-4o", instructions=..., input=[file + question], store=True)

# call 2: do NOT re-attach the file; continue the same conversation server-side
extract_response = responses.create(model="gpt-4o",
                                    previous_response_id=classify_response.id,
                                    input="Extract the following fields: [...]", store=True)
```

- **Why it is faster:** the slow part is the model ingesting the file, not generating the short answer. Chaining reuses the conversation so the file is read once.
- **Same model for both calls is essential:** in testing, chaining a gpt-5 extraction onto a gpt-4o classification took 93 s - indistinguishable from not chaining (99.7 s) - because gpt-5 re-read the file anyway. gpt-4o to gpt-4o took 15.8 s.
- **Why two calls, not one:** a merged classify-and-extract prompt was tried and reverted. Putting every document type's field list in one prompt measurably hurt classification (a file that classified correctly on its own came back NO_MATCHING_LAYOUT_TYPE). The chained design keeps both prompts unchanged and still saves the file read.
- **Defensive parsing:** if the model returns the whole taxonomy line ('document_type [lob: x]') a regex strips the annotation and retries the exact match; anything unmatched becomes NO_MATCHING_LAYOUT_TYPE rather than a guess. Markdown fences are stripped from JSON output; unparseable output is stored as {raw_response: ...} so nothing is lost.

7.4 Configuration over code

- The fields to extract come from `ref_prompt_library_final_layout.extraction_rules` keyed by (LOB, document_type). Supporting a new document type is a database row, not a deployment.
- The same table is the **taxonomy** the classifier must choose from, so the model can only answer with values the system knows how to process.
- **Broker-supplied field list:** if present, it overrides the DB list for that submission, and any new field names are written back to the DB so the knowledge base grows. The write-back is done *sequentially after* the parallel work to avoid a read-modify-write race on the same row.

7.5 Choosing the primary document

Downstream phases (sanctions, perils, risk, Guidewire) need one extraction JSON for the whole submission. Across all configured LOBs, exactly one document type contains the word 'submission' (for example Submission_Document_Commercial_Property). That is the primary document. If none exists - for instance the submission is only an ACORD - the ACORD result is used. The step also writes a tiny `lob_detection` JSON because a downstream tool expects the LOB as a Blob URL.

> **WATCH OUT**
> 'Contains the word submission' is a convention, not a schema. If asked, say you would replace it with an explicit *is_primary* flag in the taxonomy table.

7.6 Summary and chat reuse the same assets

- **Summary (Phase 7):** built from the already-extracted JSON passed as text, not by re-attaching raw files. The raw approach took 150+ s on a two-attachment submission; the text-only call is cheap. Files with no extraction output fall back to direct attach so nothing is silently omitted. The mail body is excluded on purpose.
- **Chat:** a question-answer call over the same vector store with the conversation history passed in; the instructions say answer only from the attachments and say so plainly if the answer is not there.

7.7 Resilience

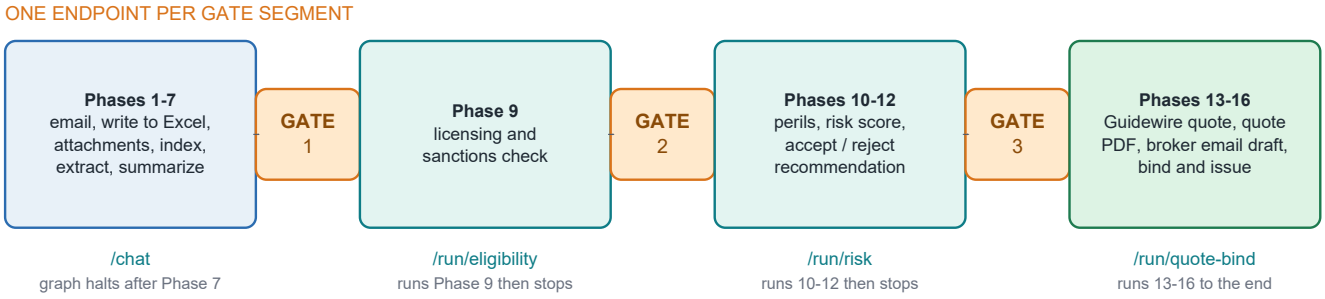
- **Rate limit retry:** five retries with delays of 2, 4, 8, 16 and 32 seconds on RateLimitError. (Add jitter if asked how to improve it.)
- **Concurrency cap of 4:** six concurrent gpt-5 calls tripped the deployment's rate limit in testing.
- **Cleanup:** in the earlier agent-version design the runtime agent version and its context blob were deleted in a finally block even on failure.

7.8 The numbers (all from comments in service.py)

What was measured	Result	Takeaway
One classify call on a real submission (gpt-5)	114 s	File ingestion dominates, not output length
Classify + extract as two independent calls, 3-file submission	111 s combined	Baseline
Same calls chained	30.5 s, matching values	File read once
gpt-5 extract chained onto gpt-4o classify	93 s (unchained gpt-5: 99.7 s)	Chaining needs the same model
gpt-4o → gpt-4o chained	15.8 s; full-size real SOV: 28.8 s, 56 of 56 fields	Chosen design
Merged classify+extract in one prompt	Reverted	Classification accuracy dropped
Combined summary re-attaching all files	150+ s for two attachments	Now text-only from extracted JSON
Six concurrent gpt-5 calls	Rate-limited	Cap at 4 plus backoff

> **SAY THIS**
> I treat latency as an engineering problem with experiments: I measured where the time went - it was reading the file, not generating - and then removed repeated reads by chaining, by reusing extracted JSON, and by running independent documents in parallel.

8. Human-in-the-loop gates deep dive



Underwriter approves in the UI -> UI calls the next endpoint -> a fresh sub-graph starts with URLs in its first message.
No in-memory graph state survives a gate: the artifacts live in Azure Blob and PostgreSQL.

8.1 The three gates

Gate	Position	Underwriter decides	On approval the UI calls	What then runs	Why here
1 Document review	After Phase 7 summary	Are the extracted facts and the LOB right?	POST /run/eligibility	Phase 9 only	Garbage in is cheapest to fix before any check or score is run on it
2 Eligibility	After Phase 9	Is the insured eligible (licensing, sanctions)?	POST /run/risk	Phases 10-12	Do not spend peril lookups and scoring on an ineligible risk
3 Risk acceptance	After Phase 12	Accept the risk at the recommended terms?	POST /run/quote-bind	Phases 13-16	Everything after this writes to Guidewire and emails the broker

8.2 How a gate works technically

- **Halt:** after each tool call, `after_tools_router` checks for a successful `summarize_attachments_mcp` in the latest tool batch. If found, the graph goes to a halt node, which only logs, and then to END. Nothing is waiting in memory.
- **Segments end themselves:** the Phase 9 and 10-12 prompts finish with 'End' after their last tool call, and the endpoint returns. The gate for those is simply 'the next endpoint has not been called yet'.
- **Resume:** the UI, after the underwriter clicks approve, calls the next endpoint with the reference id and the Blob URLs. A new graph is compiled with a narrow prompt, runs, and stops.
- **State lives outside the agent:** artifacts in Blob, workflow state in PostgreSQL and in the Workboard's own stage field. The agent process can be restarted at any time between gates.

IN THE CODE
The gates are not written in the Phoenix prompt. Its last line even tells the model to ignore any human-in-the-loop part. Gate 1 comes from the graph topology (`enable_phase1_halt=True` on the main graph), and Gates 2 and 3 come from the fact that each later segment is a separate endpoint with its own prompt and `enable_phase1_halt=False`. If asked 'where do the gates live, prompt or code?', the answer is code.

YOUR REASONING - adjust to what you actually remember
This is the answer to 'why not use LangGraph's interrupt in production?'. Gates here are policy and should not depend on whether the LLM chooses to ask. A human can take hours or days to approve, and a container restart would lose an in-memory checkpointer. The UI already owns approval state, so a stateless resume is simpler and more robust. The cost: each segment starts without the earlier conversation, so everything it needs must be in its first message.

8.3 The prototype you can also describe

```
@tool
def human_assistance(query: str) -> str:
    """Request input or approval from a human."""
    return interrupt({"query": query})            # pauses the graph, surfaces the payload

memory = InMemorySaver()
graph = create_custom_graph(model, [human_assistance] + mcp_tools, prompt, checkpointer=memory)

# first call streams until an interrupt event appears; the UI shows the query
graph.astream({"messages": [...]}, config={"configurable": {"thread_id": checkpoint_id}})
# later: resume the same thread with the human's answer
graph.astream(Command(resume=message), config={"configurable": {"thread_id": checkpoint_id}})
```

IN THE CODE
`server_hitl.py` imports `AsyncSqliteSaver` but actually instantiates `InMemorySaver`, and the production `server.py` compiles its graphs with **no checkpointer**. Do not say your production gates use a SQLite checkpointer.

8.4 interrupt() theory you should be able to explain

- `interrupt(value)` raises a special exception; the graph saves state to the **checkpointer** and returns the payload under the '__interrupt__' key.
- It requires a checkpointer and a **thread_id**; resuming is `invoke(Command(resume=...), config)` with the same thread id.
- On resume the **node that called interrupt restarts from its beginning**, with interrupt() now returning the resume value. So any side effects before the call must be idempotent.
- Multiple interrupts in one node are matched to resume values by order. A durable checkpointer (Postgres, SQLite) survives restarts; an in-memory one does not.

8.5 Preventing double runs

```
acquired = await acquire_pipeline_lock(body.reference_id)      # MCP tool backed by Postgres
if not acquired:
    raise HTTPException(status_code=409, detail="Full pipeline already triggered recently ...")
try:
    ...run 9 → 10-12 → 13-16...
except Exception:
    await release_pipeline_lock(body.reference_id)              # let a genuine retry through
    raise HTTPException(status_code=500, ...)
```

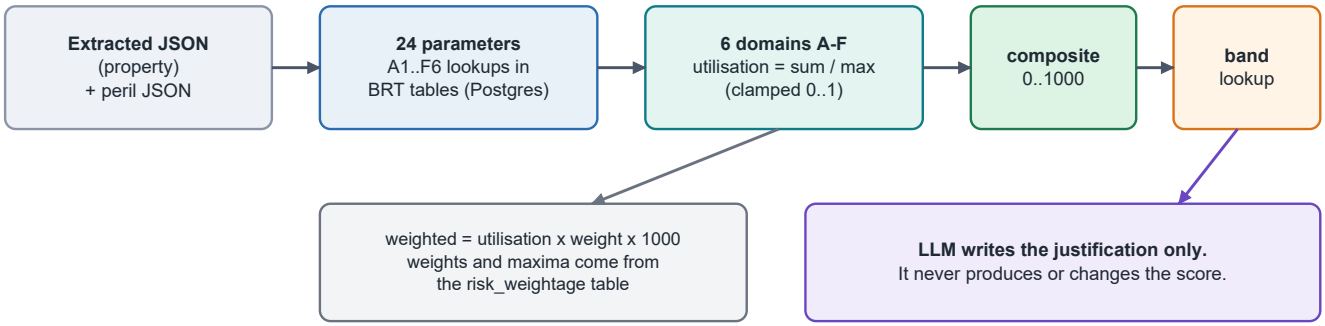
- The lock is a timestamp column (`submissions.full_pipeline_triggered_at`) with a ten-minute window. An atomic compare-and-set in SQL is what makes it a lock rather than a check-then-write race.
- **Fail-open by design:** if the lock service is unreachable the pipeline proceeds. The comment explains the trade-off: the duplicate-run race is rare, whereas blocking a legitimate run on a transient outage would be worse.
- **Release only on failure:** after success the window simply expires, so a quick re-submit is rejected rather than duplicated.
- HTTP 409 Conflict is the right status for 'this resource is already being processed'.

```
-- illustrative: the atomic claim behind such a lock
UPDATE submissions
SET    full_pipeline_triggered_at = now()
WHERE  reference_id = %s
  AND  (full_pipeline_triggered_at IS NULL
        OR full_pipeline_triggered_at < now() - interval '10 minutes')
RETURNING reference_id;      -- a row back means this caller got the lock
```

8.6 Other idempotency guards (interview-friendly list)

- Light dedup at intake: email_reader_mcp skips a repeated message id within one call, and Phase 4 reports ALREADY_INDEXED instead of re-uploading. (A dedicated attachment_checker_mcp is registered but the v103 prompt does not call it.)
- Incremental indexing keyed on file names in Postgres.
- Webhook ingestion on the UI side is idempotent on the email message id.
- The Workboard's stage flow is forward-only, so a late or repeated message cannot move a completed submission backwards.
- Phase endpoints take a reference id and URLs, so re-running a segment overwrites its own outputs instead of corrupting shared state (Guidewire is the exception - see Part 10).

9. Risk scoring, perils and decisioning



9.1 Principle: LLM explains, rules decide

The composite score is computed by deterministic code from lookup tables. The LLM receives the raw inputs, scores, domain sums and rating and writes a short, regulator-friendly justification. It cannot change the number. That separation is the most important thing to say about risk scoring: **auditable, reproducible, tunable by an underwriter editing tables, no prompt in the loop.**

9.2 The algorithm

- **Extract** raw values from the property JSON and the peril JSON with a fallback list of key names per parameter (labels differ between documents) after flattening one level of nesting.
- **Look up points** for each parameter in its table: exact-match tables (construction class, occupancy, flood zone, sprinkler, ...) and **range tables** using PostgreSQL `int4range` containment (`value <@ range`) for ages, vacancy rates, months since inspection and so on. Missing or failing lookups score 0 and the transaction is rolled back so later lookups still work.
- **Aggregate per domain** A-F: utilisation = domain sum divided by the domain maximum, clamped to 0..1 (credits cannot push below zero).
- **Weight:** weighted = utilisation × domain weight × 1000. The composite is the sum, in the range 0-1000. Maxima and weights are read from a `risk_weightage` table.
- **Band:** the composite is looked up in `risk_score_band` to return band, rating, underwriter action, premium modifier and required conditions.

Domain	Meaning	Parameters	Max	Weight
A	Construction and building systems	A1 construction class, A2 building age, A6 roof, A7 HVAC, A8 electrical, A9 plumbing	250	25%
B	Occupancy	B occupancy type, B5 vacancy rate	200	20%
C	Geographic / catastrophe	C2 FEMA flood zone, C4 wildfire, C5 wind zone, C6 earthquake PML, C7 hail	200	20%
D	Loss history / financial	D1 prior carrier departure, D6 BI period adequacy, D8 years in business	150	15%
E	Fire protection and life safety	E1 sprinklers, E3 fire alarm, E5 security, E8 backup power, E6 last fire inspection	120	12%
F	Regulatory / risk management	F1 building code compliance, F4 risk management program, F6 claims litigation	80	8%

> **SAY THIS**
> It maps neatly to the underwriting acronym COPE - Construction, Occupancy, Protection, Exposure - plus loss history and regulatory factors.

Worked example (illustrative numbers)

Domain	Sum of points	Max	Utilisation	Weight	Weighted score
A	140	250	0.56	25%	140
B	90	200	0.45	20%	90
C	120	200	0.60	20%	120
D	60	150	0.40	15%	60
E	48	120	0.40	12%	48
F	40	80	0.50	8%	40
Composite					**498**

A composite of 498 falls in the 350-500 band: **Accept with Conditions**, human review required.

9.3 Accept / reject bands

Composite score	Decision	Human review?
≤ 200	Auto-Accept	No
200 - 350	Accept	No
350 - 500	Accept with Conditions	Yes
500 - 650	Senior Underwriter Review	Yes
650 - 800	Refer to Specialty	Yes
800 - 1000	DECLINE	No (but the underwriter still sees it at Gate 3)

- Bands are objects with lower/upper bounds and **inclusivity flags**, so adjacent bands never overlap or leave gaps for float scores. A validation method checks ordering and overlap.
- A score no band covers falls back to 'Needs Review' - fail towards a human.

> **WATCH OUT**
> The accept/reject bands are defaults in code, while the rating bands come from the risk_score_band table. Two sources of truth for similar thresholds is a smell; mention you would unify them.

9.4 Peril data capture (Phase 10)

- Read the property JSON, find the address section (the key is matched after normalising away case, spaces and punctuation, so 'Name And Mailing Address:' still matches).
- **Geocode** the address with the OpenWeather geocoding API into latitude and longitude.
- **Earthquake:** query the USGS event API for the count of events within 10 km over the last 5 years.
- **Other perils** (flood zone, wind zone, wildfire score, hail score, ISO PPC rating and so on) come from a PostgreSQL `PERIL_RISK_DATA` table through a nearest-neighbour query ordered by squared latitude/longitude distance, LIMIT 1.
- Output is a peril JSON in Blob; the risk scorer reads it next.

> **WATCH OUT**
> Squared distance on raw degrees is a planar approximation with no spatial index. Fine for a demo-scale table, but say you would use PostGIS (geography type, GiST index, ST_DWithin / KNN) at scale.

9.5 What else Phase 11 does

- Writes the full breakdown (individual scores, domain sums, weighted contributions, composite, rating, justification) as JSON to Blob.
- Persists per-domain results to the domain scoring database and appends a row to an Excel workbook that the Workboard monitors - this is how the UI learns the result without the agent calling it.

- The justification uses a Phoenix-hosted prompt, a senior-underwriter system message and instructions to stay strictly within the provided data and not reveal internal formulas.

WATCH OUT
Two details to verify before you present them as fact: the domain configuration is loaded once at import time (a table change needs a restart), and parameter C10 (ISO PPC rating) is looked up but is not in the domain C member list, so it appears not to contribute to the composite.

10. Guidewire integration (Phases 13 and 16)

10.1 Quote flow (commercial property)

Step	Call	Notes
0	Read extraction JSON from Blob; map to a canonical model	mappers/document_mapper.py (commercial) and homeowners_mapper.py (residential)
1	POST /account/v1/accounts	Account payload built from the canonical model
2	POST /job/v1/submissions	Product id forced to GoCommercialProp for commercial
3	POST /job/v1/jobs/{jobId}/lines/GOCommercialLine/locations	Location payload (address, building facts)
4	POST .../coverages (one per coverage)	Coverage list from a coverage builder
5	POST /job/v1/jobs/{jobId}/quote	Returns quote with jobNumber
6	Save quote JSON to Blob; insert (submission_id, job_number) into guidewire_data	So other phases and the UI can map a submission to its Guidewire job

10.2 Bind and issue (Phase 16)

- `POST /job/v1/jobs/{jobId}/bind-and-issue` with the job id carried from Phase 13; the response is saved to Blob.
- Only reachable after Gate 3. Binding is the irreversible commitment, which is why it sits behind a human approval.

10.3 Design points worth saying

- **Mapping layer:** extraction JSON → canonical model → payload builders. Guidewire's schema never leaks into the extraction code, and a new LOB means a new mapper, not changes everywhere.
- **Explicit step errors:** each step returns `{status: False, error: 'Failed to create job: ...'}`, so the agent and the logs tell you exactly which call failed.
- **Separate handlers** for commercial and homeowners (RESIDENTIAL PROPERTY) lines.

> **WATCH OUT**
> The quote flow is not idempotent: a retry after a late failure creates another account and job in Guidewire. A fix: store the account id and job id as soon as they are created, and look them up before creating again.

11. Reliability, observability, security, deployment

11.1 Mechanisms already in the code

Failure mode	Mechanism	Where
LLM rate limiting (429)	Exponential backoff, 5 retries; concurrency cap 4	Foundry service
Slow tool calls through proxies	SSE keep-alive every 20 s; 120 s connect and 600 s read timeouts	Agent /chat, MCP client config
Runaway agent loop	recursion_limit 20 / 75 / 250 depending on segment	Each endpoint
Double trigger of a run	Postgres TTL lock with HTTP 409; release on failure	/run/full-pipeline
Wrong tool or phase order	Narrow per-segment prompts with GUARDRAILS and explicit tool names	Segment prompts
LLM label drift	Taxonomy from DB, exact match, regex fallback, NO_MATCHING marker	Foundry service
Unparseable JSON from the model	Fence stripping, raw_response fallback	Foundry service
Non-insurance emails	NOT_INSURANCE_SUBMISSION marker ends the run early	LOB match
Cross-submission data leakage	One vector store per submission	Foundry service
Re-indexing cost on follow-up emails	Diff Blob against recorded files, upload only new	KB table
Bad prompt release	Pinned prompt version ids; one-line rollback	server.py
Silent tool-registration mistakes	operation_id whitelist (a known gotcha)	test_main.py
Debuggability	Per-execution append-blob logs; structured AGENT_ERROR record with last tool	Agent
Bad scoring lookups	Missing value → 0 points; per-parameter try/except with rollback	Risk scorer
Score outside configured bands	Falls back to Needs Review	Accept/reject

11.2 Observability

- **Prompts:** Arize Phoenix hosts and versions them. An OpenTelemetry tracer is registered, though in the version I read the LangChain auto-instrumentation line is commented out. Describe Phoenix primarily as prompt management unless you know tracing is on in your deployed build.
- **Logs:** structured step-by-step execution logs in Blob; container stdout (visible with the container platform's log command) for application errors.
- **Metrics you would add:** per-tool latency histograms, token usage per submission, extraction field-fill rate, gate wait times, approval/override rates.

11.3 Deployment

- Docker images, deployed to Azure Container Instances inside a virtual network; the tool server is reached over an internal DNS name. Secrets (connection strings, API keys) are meant to come from Azure Key Vault and environment variables, never code.
- Local development uses plain virtual environments and the three commands in 5.1.

11.4 Security: what to say and what to improve

Topic	Today (from the code)	What you would do
Network exposure	Internal VNet; CORS configured as allow-all	Restrict origins; put an API gateway in front
AuthN/Z between components	None visible in agent or tool code	Managed identity / Entra ID tokens, per-tool scopes
Azure access	AzureCliCredential used for Foundry client	Managed identity in containers; no CLI credential
Prompt injection	Documents are untrusted input; extraction prompts demand JSON; irreversible actions are behind gates	Treat tool outputs as data, strip instructions, separate extraction from action, allow-list tools per segment
PII	Insured names/addresses flow through logs and Blob	Mask in logs, retention policy, encryption, access reviews, data residency
Secrets	Env vars / Key Vault	Rotate, scope, no secrets in prompts or logs
Auditability	Execution logs, Blob artifacts, UI stage history	Immutable audit store; who approved what and when; link to prompt version

12. Engineering stories (STAR)

Ten stories with real details from the code. Pick two or three as your 'go-to' answers for 'tell me about a hard problem' and keep the rest for follow-ups. Every number below is from a comment in the repository - say 'in our testing'.

> **WATCH OUT**
> These are written in the first person so they are easy to rehearse, but they describe what the *team* did according to the code. Tell a story as 'I' only if you personally did it; otherwise say 'we' and describe your part. If you were not involved in one at all, skip it rather than borrow it.

1. Cutting extraction time from 111 s to 30 s

Situation	Classifying and extracting each document took minutes on real submissions; a single classify call took 114 s even though its entire output was one word.
Action	Profiled and found that reading the file, not generating text, dominated. Tried merging classify and extract into one prompt - classification accuracy dropped, so I reverted it. Then chained the second call to the first with previous_response_id so the file is read once, discovered chaining only helps when both calls use the same model, and standardised on gpt-4o. Ran independent documents in parallel.
Result	Two calls: 111 s → 30.5 s on a three-file submission; 15.8 s chained on a smaller case; a full-size SOV extracted 56 of 56 fields in 28.8 s.
Lesson	Measure before optimising; run the accuracy experiment and be willing to revert; know why an optimisation works so you know when it will not.

2. The LOB the agent translated once and forgot the next time

Situation	Downstream tools required the exact strings RESIDENTIAL PROPERTY or COMMERCIAL PROPERTY, but the taxonomy uses codes like 'home' and 'commercial'. The prompt expected the agent to translate.
Action	Observed live that 'home' was translated correctly once, while 'commercial' passed through untranslated and failed with 'Unknown LOB'. Moved the mapping into code inside process_submission_mcp, returned a canonical detected_lob (None for LOBs not wired to quoting), updated the prompt to pass it verbatim and to end gracefully when it is null.
Result	Deterministic mapping; no more per-run luck. Pinned prompt v103.
Lesson	If a transformation has one right answer, do not ask an LLM to do it. Prompts should carry intent, code should carry contracts.

3. Rate limits under parallel load

Situation	Running several documents concurrently produced HTTP 429s from the model deployment.
Action	Added exponential backoff (2, 4, 8, 16, 32 s, five retries) in a single wrapper used by every Responses API call, and capped concurrency at four after six concurrent calls tripped the limit in testing.
Result	Transient 429s are absorbed; bursts are smaller, so fewer retries and less time lost to waiting.
Lesson	Fix both sides: retry for resilience, concurrency caps to avoid causing the problem. Add jitter at higher scale.

4. Two pipeline runs for one submission

Situation	A repeated trigger (double click or retry) could start overlapping Phase 9-16 runs for the same submission.
Action	Added an atomic claim in PostgreSQL behind an MCP tool with a ten-minute window; the endpoint returns HTTP 409 on a duplicate. Chose fail-open when the lock service is unreachable, and release on failure so a real retry is not blocked.
Result	Overlapping runs are rejected; a transient lock outage does not block legitimate work.
Lesson	Idempotency is a design choice with explicit trade-offs: decide whether you fail open or closed and write down why.

5. A prompt release that crashed prompt loading

Situation	A new prompt version (v101) stored its content as a plain string instead of a list of text blocks. The loader indexes into the first block, so every request failed with 'string indices must be integers'.
Action	Because prompt versions are pinned by id in code, I reverted to the previous id immediately and published v102 with the correct shape and the same text.
Result	Short outage window, one-line rollback, documented in the code comment for the next person.
Lesson	Version and pin prompts like code. Add a schema check to the publish step.

6. Preventing one submission leaking into another

Situation	A shared vector store would have been simpler, but I needed each query scoped to one submission.
Action	Tested Foundry's file attributes and filters; they came back None after upload and there was no update endpoint. Switched to one vector store per submission, tracked in PostgreSQL, with incremental indexing so a growing thread only uploads new files.
Result	Isolation by construction instead of by filter; no cross-contamination possible.
Lesson	Test a vendor feature before you build on it, and choose designs where a failure cannot leak data.

7. One submission, several document types

Situation	Early layout detection ran at submission level, but a single submission contains an ACORD form, an SOV and loss runs.
Action	Kept LOB detection at submission level (it needs all documents together) and moved layout classification and extraction to the document level, attaching each file directly. Collapsed three LLM-driven phases into one deterministic tool.
Result	Each document gets its own classification and its own extraction JSON; the agent makes one tool call instead of three.
Lesson	Choose the right grain for each decision. Fewer LLM-chosen steps means fewer failure points.

8. Streams that died behind proxies

Situation	Tool calls take 50-72 s and whole runs take minutes. Proxies and load balancers cut connections that are silent for too long.
Action	Ran the graph in a background task feeding a queue; the response generator emits an SSE comment every 20 s when idle. Set generous but explicit MCP timeouts (120 s connect, 600 s read). Wrapped failures in a structured error with the last tool invoked and elapsed times, and wrote it to a dedicated container.
Result	Streams stay alive; when something does fail, the error tells you which tool and how long it ran.
Lesson	For long-running LLM workflows, transport reliability is part of the design, not an afterthought.

9. Prompt-tool contract drift

Situation	A prompt referred to a retrieval tool by the wrong name, and another phase used the raw layout label instead of the matched document type.
Action	Corrected both in new prompt versions, added regex normalisation in code for layout labels, and started treating tool names as a public contract.
Result	Fewer silent wrong-tool calls.
Lesson	Add a test that every tool named in a prompt exists in the registered tool list, and fail the build if not.

10. A 150-second summary that did not need to exist

Situation	The combined summary re-attached every raw attachment and was the single most expensive step.
Action	Realised extraction had already distilled the same content into JSON. Fed that JSON as text, with a fallback that attaches the raw file only when no extraction exists, and excluded the email body on purpose.
Result	A cheap text-only call; no silent omissions.
Lesson	Reuse upstream artifacts; ask 'what do I already know?' before paying for another read.

13. Honest gaps - and how to say them well

Interviewers are not looking for a perfect system; they are looking for someone who knows exactly where the edges are. Raise these yourself when they fit - it earns more credibility than being caught. Each row says what the code does, a phrasing, and the fix.

Topic	What the code actually does	How to say it	The fix
Sanctions screening	Compares the extracted insured name, case-insensitively and exactly, with a list in a CSV	'A baseline exact-match screen; the design allows a real provider to slot in.' Avoid saying 'OFAC screening'	Fuzzy and transliteration-aware matching, official list feeds (OFAC SDN, UN, EU), match scores, audit log
Licensing / eligibility rows	Eligibility result columns are copied from a template workbook row chosen by the sanction flag and LOB	'Eligibility output is stubbed at this stage; sanctions is real logic.'	Integrate state licensing data and business rules from a rules table
LLM as orchestrator	The agent follows a long prompt; the router matches the words 'End' and 'Continue' as substrings	'Prompt-driven sequencing with deterministic guards at gates; the next step is an explicit graph.'	Explicit node per phase, structured output or a state flag instead of string matching
All tools bound everywhere	Every sub-graph binds all 34 tools; the prompt restricts use	'Prompt-level least privilege.'	Bind only the tools each segment needs
No production checkpointer	Graphs compile without a checkpointer; resume relies on stateless endpoints and idempotent phases	'Gates are stateless resume points by design.'	Durable checkpointer (Postgres) plus interrupt nodes for in-graph approvals
Guidewire retries	Quote flow creates a new account and job each time it runs	'Not idempotent yet.'	Persist account and job ids immediately; look up before create
Pipeline lock	Fail-open; timestamp window, released only on failure	'A deliberate availability-over-strictness choice.'	Advisory locks or a job table with unique constraint
Excel hand-off	Read-modify-write of a workbook in Blob (download, append, overwrite)	'A pragmatic integration contract with the UI.'	ETag-conditional writes, or a queue or database table
Security	CORS allow-all, no auth on agent and tool endpoints visible in code, AzureCliCredential for Foundry	'Relies on private networking today.'	Managed identity, API gateway, per-tool scopes
Logging and PII	Execution logs write full message content to Blob	'Needs masking and retention rules.'	Redaction, retention, access control
Peril lookup	Nearest neighbour by squared lat/lon distance, table scan	'Good enough for a dataset this size.'	PostGIS geography with a GiST index; haversine
Risk configuration	Domain weights loaded at import time; C10 is looked up but not in the domain-C sum; thresholds in two places	'Worth a cleanup pass.'	Hot-reload config; fix C10; one source of truth
Evaluation	No automated evaluation harness found in the code I reviewed	'We validate by underwriter review at Gate 1; next is a golden set with regression runs.' (Adjust if you do have one.)	Gold documents, field-level precision/recall, CI gate, Phoenix experiments
Tracing	OTel tracer registered, LangChain auto-instrumentation commented out	'Phoenix is mainly our prompt registry today.'	Enable instrumentation; trace per submission
Per-request overhead	Fetches tools and the Phoenix prompt on every /chat	'Simple and always fresh.'	Cache with a short TTL and a version check
Legacy tools	Chroma and pgvector era tools are still registered, so the model can see them; only the prompt keeps it away (v103 says 'do not call risk_type_checker_mcp')	'Kept for rollback; due for removal.'	Remove from the whitelist; archive the code

Topic	What the code actually does	How to say it	The fix
Prompt-tool contract drift	The v103 prompt names 'create_draft' (real id email_draft_mcp) and an '&' tool name that cannot be a function name; two cross-references are stale after renumbering	'The model maps near-miss names today; the fix is a startup check that every tool named in the prompt exists.'	CI or startup validation of prompt tool names against get_tool_list; fail the deploy on a mismatch
Master versus segment prompts	Phases 9-16 exist twice: in the Phoenix prompt and as constants in server.py, with different details (Continue versus End, phase references, tool names)	'Segment prompts are the corrected, narrower copies; the master tail is legacy.'	One source of truth: store segment prompts in Phoenix too and generate them from the same phase definitions
Duplicate detection	attachment_checker_mcp is registered but the current prompt never calls it; dedup is a repeated-message-id skip inside email_reader and idempotent indexing	'Idempotent by design at the indexing step; a dedicated attachment duplicate check exists but is not in the flow.'	Hash-based dedup keyed on message id plus attachment checksum, enforced in code
Primary document rule	Document type containing the word 'submission' is chosen	'A convention I would make explicit.'	is_primary flag in the taxonomy table
Structured output	JSON requested in the prompt and parsed defensively	'Works, but not schema-enforced.'	JSON-schema structured outputs plus a repair-and-retry step

WATCH OUT
On your resume: change 'OFAC sanctions screening' to 'sanctions-list screening', and replace 'LangGraph checkpointing (AsyncSqliteSaver)' with the gate description from Part 8. Both are easy to defend once worded accurately; the original wording is not.

14. Q&A bank

Read the question, answer out loud for 45-60 seconds, then compare. Answers are written the way you would speak them, not as textbook paragraphs. Where an answer depends on your own memory, it says so.

A. Project overview and ownership

Q1. Walk me through the project.

It automates property underwriting for a P&C insurer. A broker's email comes in; a LangGraph agent drives 16 phases through 34 MCP tools. It indexes the attachments into an Azure AI Foundry vector store, classifies and extracts every document, summarises, screens the insured, captures perils, scores the risk, and then gets a Guidewire quote and binds the policy. Three human gates sit at document review, eligibility and risk acceptance. The two things I would dwell on are the extraction layer, which evolved from ChromaDB to pgvector to Foundry, and the gate design.

Q2. What problem does it solve and for whom?

Underwriters spend hours per submission reading emails and attachments, re-keying into Guidewire and running checks. The system does the reading, extraction, screening and scoring, so the underwriter spends time on judgement. It also makes every step reproducible and auditable, which matters in a regulated business.

Q3. What was your role?

[Fill in.] A good shape: 'I owned the [extraction pipeline / agent orchestration / MCP tool layer] end to end - design, implementation, performance tuning - and worked with [teammates] on [Guidewire / UI]. The UI was generated with Replit, so I stayed on the backend.'

Q4. How big is it?

Sixteen phases, 34 exposed MCP tools across about two dozen modules, three human gates, three processes (agent, tool server, UI), five LOB codes classified of which home and commercial property are wired through Guidewire, and four Azure services at the centre: Foundry, Document Intelligence, PostgreSQL and Blob.

Q5. Why an agent instead of a fixed workflow?

The inputs vary a lot - non-insurance emails, missing documents, unsupported LOBs, templated and free-form documents - and the agent can handle the exceptions and write natural-language summaries and emails. But I will be honest: where the flow is fixed we have progressively moved decisions into code - gate halts, LOB mapping, scoring, banding. It is a hybrid, and if I started again I would make the sequencing an explicit graph and keep the LLM for reading and writing.

Q6. What was the hardest technical problem?

Extraction at the document level: submissions contain several different document types, each needs the right fields, the latency was minutes, and accuracy could not drop. We fixed it with per-document classification against a database taxonomy, direct file attach instead of retrieval, chaining two calls to read each file once, and parallelism with backoff. That took the classify-plus-extract from 111 s to about 30 s in our tests.

Q7. What would you do differently?

Four things, in order:

- An explicit graph with a durable checkpointer instead of prompt-driven sequencing and string-matched routing.
- An evaluation harness: a gold set, field-level precision and recall, regression runs on every prompt change.
- Per-segment tool allow-lists, structured outputs, and managed-identity auth.
- Idempotent Guidewire calls and fuzzy sanctions matching.

Q8. How do you know it works?

Today: confidence scores from the ACORD model, underwriter review at Gate 1, and spot-checks against source documents. I would add a gold dataset and automated regression checks. [Quote only the metrics you can defend.]

Q9. Which lines of business does it support?

Home, which maps to RESIDENTIAL PROPERTY, and commercial, which maps to COMMERCIAL PROPERTY, are wired end to end. Management liability, new submission and political are classified and extracted but not yet connected to quoting, and the pipeline ends gracefully for them.

Q10. Who uses it, and what does the user see?

Underwriters use a Workboard that shows submissions, extracted documents, eligibility and risk results and lets them approve at each gate. I will keep my answers to the backend because that UI was generated with Replit; it talks to the agent through three approval endpoints and reads results from Blob and webhooks.

B. Architecture and design decisions

Q11. Why MCP instead of ordinary LangChain tools?

One tool surface serves the agent, the chat feature, tests and curl. Tools redeploy independently of the agent, heavy Azure dependencies stay in the tool server, and the agent discovers tools at request time. The cost is a network hop per call and a bigger tool list in the model's context.

Q12. Why Azure Blob as the integration bus?

It decouples producers from consumers, it is durable, cheap, auditable, and every phase's output is a file with a URL that the next phase can take. The UI monitors it and also gets webhooks. The trade-offs are polling latency and fragile patterns such as appending to an Excel workbook; a queue or event grid and a database table would be cleaner at scale.

Q13. Why are the agent and tool server separate processes with separate virtual environments?

Different dependency sets and different change rates. It also enforces the boundary: the agent can only call tools over HTTP, not import their internals.

Q14. How does the agent know which tools exist?

On every request it connects with MultiServerMCPClient, calls get_tools, and gets LangChain tool objects whose schemas come from the FastAPI route definitions. Those are bound to the model with bind_tools and executed by ToolNode.

Q15. What does the LLM decide and what is hard-coded?

The LLM decides which tool to call next within a segment, writes the broker email, and does the reading: LOB, classification, extraction, summary.

Code decides gate halts, LOB mapping, taxonomy validation, scoring, banding, locks, retries and timeouts. Anything with one right answer is in code.

Q16. How is state passed between phases and across gates?

As text in the messages - mostly Blob URLs - and through artifacts in Blob and rows in PostgreSQL. Each gate resumes with a fresh sub-graph whose first message carries the reference id and URLs. Nothing important lives only in agent memory.

Q17. Why separate sub-graphs per gate instead of one graph with interrupts?

Gates are policy, so they should be deterministic, not the model's choice. A human may take hours, and a container restart would lose an in-memory checkpointer. The UI already owns approval state. Narrow segment prompts are also safer and cheaper. The cost is rebuilding context in each first message.

Q18. How would you scale this ten times?

- Put submissions on a queue (Service Bus); stateless workers pull a submission and run a segment.
- Scale the agent and tool server horizontally; they are stateless apart from Blob and Postgres.
- Budget model throughput: per-deployment rate-limit budgets, a shared limiter, separate deployments for batch and interactive.
- Connection pooling for Postgres, per-tenant quotas, circuit breakers on Guidewire and Foundry.
- Idempotent steps so retries are safe; dead-letter queue for poison submissions.

Q19. How do you handle long-running tool calls?

Async endpoints, explicit timeouts (120 s connect, 600 s read), SSE keep-alive comments every 20 s so proxies do not drop the stream, and a background task feeding a queue so the stream keeps flowing even while a tool blocks.

Q20. What happens if the MCP server or Foundry is down mid-run?

A failed tool returns an error ToolMessage; the gate detector ignores errors, so it will not falsely advance. Rate-limit errors are retried with backoff. Anything else fails the segment with a structured error record naming the last tool, and the segment can be re-run because phases write their own artifacts. The exception is Guidewire creation, which I would make idempotent.

Q21. How do you avoid duplicate processing?

Several layers, and I would be honest that the intake dedup is light.

- email_reader skips a repeated message id within a call, and Phase 4 is idempotent: incremental indexing keyed on file names returns ALREADY_INDEXED instead of re-uploading.
- A Postgres TTL lock with HTTP 409 protects the full-pipeline endpoint, the UI webhooks are idempotent on message id, and the stage flow is forward-only.
- There is a dedicated attachment duplicate checker tool, but the current prompt does not call it; I would enforce hash-based dedup in code.

Q22. How do you keep prompts and tools in sync?

Pinned prompt versions plus treating tool names as a contract, but today nothing enforces it. We had two incidents of drift - a wrong tool name (v83) and an extraction phase using the wrong field (v84) - and the current prompt still has near-miss names: 'create_draft' for the tool registered as email_draft_mcp and an ampersand in the sanctions tool name.

The fix is a startup or CI check that extracts the tool names from the prompt and fails if any is missing from the list the MCP client returns. That is the first thing I would add.

Q23. Walk me through your agent prompt.

It is one system prompt stored in Phoenix and pinned by version id. It has a role line, a response protocol (answer exactly Continue or End where a phase ends with a verdict), sixteen numbered phases and six guardrails.

- Each phase names the tool, where every argument comes from (mostly the Phase 5 result) and what to do on failure.
- Branch rules are explicit: not insurance, no index, not an insurance submission, null detected_lob, missing jobId all end the run.
- The guardrails are fail-closed: if anything is missing, unclear or circular, answer End.

The design lesson is that the prompt is the wiring between tools, so it needs the same discipline as an interface: versioned, tested and checked against the tool list.

Q24. Why are there prompts both in Phoenix and in the code?

The Phoenix prompt is the master definition of the whole flow, edited and versioned by the team. After Gate 1 each approval starts a separate narrow run, and those segment prompts (Phase 9, Phases 10-12, Phases 13-16) are constants in server.py. They are short, rarely change and carry tighter guardrails. The weakness is two copies of the same phases that can drift, which is why I would move them into Phoenix as well.

C. LangGraph

Q25. What is LangGraph and why use it over a LangChain AgentExecutor?

LangGraph models an agent as a state machine: typed state, nodes, edges and conditional edges, with cycles, persistence, interrupts and streaming. You get explicit control over flow, which the opaque AgentExecutor loop does not give. That control is what lets us put gate halts in code. Note that LangChain's current prebuilt agents are themselves built on LangGraph.

Q26. Explain state, nodes, edges, conditional edges and reducers.

State is a typed dict the graph passes around. Nodes are functions that take state and return updates. Edges say what runs next; conditional edges call a routing function that returns the next node name. A reducer says how an update merges into a key - for messages, add_messages appends and updates by id instead of overwriting.

Q27. Describe your graph.

Three nodes - agent, tools, phase_1_halt. START goes to agent. After agent, a router sends to tools if there are tool calls, or ends if the reply is 'End'. After tools, a second router checks whether the summary tool just succeeded - if so go to the halt node and end, otherwise back to agent. I compile one graph per gate segment with a different prompt.

Q28. How does your router work and what is risky about it?

It checks the last AI message: tool calls → tools; content containing 'End' → end; 'Continue' → loop. The risk is substring matching on model output - 'Endorsement' contains 'End'. The prompts demand exactly the single word, but a robust version uses structured output or a state flag.

Q29. What does ToolNode do and how are tool errors handled?

It executes the tool calls from the last AI message - concurrently where there are several - and returns ToolMessages. Exceptions are caught and returned as error ToolMessages by default, so the model can see the failure. My gate detector checks the message status and ignores errors.

Q30. What is recursion_limit?

The maximum number of super-steps before LangGraph raises GraphRecursionError. It is a circuit-breaker for loops. I use 20 for a single-tool segment, 75 for multi-tool segments and 250 for the first segment with the most phases.

Q31. What is a checkpointer and a thread_id? Why is none used in your production graphs?

A checkpointer saves state after each step, keyed by thread_id, which enables resume, time travel and interrupts. Production uses none: gates are stateless resume points, with state in Blob and Postgres, so I do not need to persist graph memory across approvals. For in-graph approvals I would use a durable Postgres checkpointer.

Q32. Explain interrupt() and Command(resume).

interrupt(value) pauses the graph, saves state, and surfaces the value. You resume the same thread with Command(resume=answer), and interrupt() returns that answer. The node that called interrupt re-runs from its start on resume, so side effects before the call must be idempotent. It needs a checkpointer.

Q33. What streaming modes does LangGraph have?

values (full state after each step), updates (only changes per node), messages (token-level), custom (your own events) and debug. I use updates, so each node's output arrives as it happens and I can turn it into SSE events and logs.

Q34. How would you redesign this as an explicit graph?

- One node per phase; edges follow the pipeline; conditional edges for not-insurance, sanctioned, unsupported LOB.
- LLM only in nodes that read or write language: classify, extract, summarise, draft email.
- Interrupt nodes for the three gates with an AsyncPostgresSaver and thread_id = reference id.
- Per-node retry policies, a typed state with URLs rather than message strings, and a Send fan-out for per-document extraction.

Q35. How would you do parallel fan-out in LangGraph?

Use the Send API in a conditional edge to dispatch one task per item, with a reducer on the results key to merge them. In my project the fan-out is a ThreadPoolExecutor inside the tool because the Foundry SDK calls are synchronous; in an explicit graph I would use Send.

Q36. How do you test a graph like this?

Unit-test routers and the gate-detection function with hand-built message lists. Test nodes with a fake chat model that returns scripted tool calls. Integration-test against recorded MCP responses. Run a few real end-to-end submissions in a staging environment before changing prompts.

Q37. What does bind_tools do, and can the model call tools in parallel?

It converts the tools into the provider's function-calling schema and attaches them to each request. The model returns tool_calls; models may return several at once, and ToolNode runs them concurrently. You can force a choice with tool_choice.

D. Model Context Protocol

Q38. What is MCP and what problem does it solve?

An open protocol, built on JSON-RPC, for connecting AI applications to tools and data. Instead of every app writing custom integrations for every tool, a server exposes capabilities and any client can discover and call them. It standardises discovery, schemas and invocation.

Q39. What are MCP's primitives and transports?

Primitives: tools (model-invoked actions), resources (read-only context the app attaches), prompts (reusable templates the user picks). Transports: stdio for local processes and Streamable HTTP for remote servers (the older HTTP+SSE style is superseded). I use Streamable HTTP because the tool server runs as a remote service.

Q40. How did you expose your tools?

Each capability is a FastAPI route with an operation_id. FastApiMCP wraps the app, restricts it to an include_operations whitelist, and mounts streamable HTTP at /mcp. FastAPI's OpenAPI schema becomes each tool's input schema. Gotcha: if an operation_id is missing from the whitelist the tool silently does not exist.

Q41. How does the LLM know a tool's parameters?

From the JSON schema the client fetched with tools/list, which derives from the route's pydantic models, summary and docstring. Clear names and descriptions are part of the prompt, so they affect tool selection accuracy.

Q42. MCP versus function calling versus REST?

Function calling is the model-level mechanism: the model emits a structured call. MCP is a protocol for discovering and invoking tools across processes, and it still uses function calling underneath. REST is the plain interface; in my project the same app is both - REST for deterministic calls like the pipeline lock, MCP for tools the LLM chooses.

Q43. What are the security concerns with MCP?

Authentication and authorisation per tool, least privilege, and above all **indirect prompt injection**: email bodies and attachments are untrusted text that the model reads. Mitigations: treat tool output as data, constrain extraction prompts to JSON, give the model only the tools it needs per segment, and put every irreversible action - bind, send - behind a human gate.

Today my environment relies on private networking; I would add managed-identity tokens and per-tool scopes.

Q44. How do you manage tool versions and compatibility?

Stable operation_ids, additive changes, legacy tools kept for rollback but eventually removed, pinned prompt versions that name tools, and a test that fails if a prompt references a missing tool.

Q45. What timeouts did you configure and why?

120 s to connect and send, 600 s between SSE chunks. The slowest tool call took 50-72 s in the pgvector era, and containers can cold-start, so 600 s gives about ten times headroom. Better than hanging forever; worse than a smaller number if you want fast failure - tune it per tool.

Q46. How do you debug a misbehaving tool call?

The per-execution log shows every node, tool call and result; the AGENT_ERROR record has the last tool, elapsed time and traceback; I can call the same route directly over REST with curl; and I check the include_operations whitelist and the prompt's tool name.

Q47. Can an agent use multiple MCP servers?

Yes - MultiServerMCPClient takes a dictionary of servers and merges their tools. Mine has one server with 34 tools. With more, I would namespace tool names to avoid collisions.

E. RAG, vector stores and the evolution

Q48. Explain the move from ChromaDB to pgvector to Azure AI Foundry.

Chroma: fastest way to validate extraction - local, no infrastructure, 800/100 chunks - but it lived on the server's disk, wasn't shared or isolated.

pgvector: durable and shared in Azure PostgreSQL, but we owned chunking, embedding, indexing and retrieval, and calls took 50-72 s with spreadsheets chunking poorly.

Foundry: managed vector stores and File Search, Code Interpreter for spreadsheets, per-submission isolation, and for classification and extraction we skip retrieval entirely and attach each file directly. Each step removed a pain we measured.

Q49. Why 800-character chunks with 100 overlap?

A middle ground: big enough to keep a field with its label and neighbouring context, small enough for precise retrieval and cheap prompts. Overlap prevents splitting a value from its label at a boundary. It was a starting point we validated empirically, not a magic number. [Say what you actually tuned.]

Q50. Why 3072-dimensional embeddings?

That is the output size of the large embedding model we used. Higher dimensions can capture more nuance at the cost of storage and index size and slightly slower search; many models allow truncating dimensions if cost matters.

Q51. Why was the pgvector path slow and how would you optimise it?

The slowness wasn't only the database. A retrieval-then-LLM chain per field, large prompts and cold starts add up. I would add an HNSW index, pre-filter by submission id, retrieve once for many fields, cache embeddings, and run fields in one structured call. In the end, removing the retrieval step was the bigger win.

Q52. When would you choose pgvector over a managed vector store?

When you need SQL joins and filters with the vectors, tight control of data residency and cost, hybrid keyword and vector search in one engine, or want to avoid vendor lock-in. Managed stores win when the team is small and you want to avoid owning chunking and indexing.

Q53. How do you stop one submission's documents leaking into another's answers?

One vector store per submission, tracked in Postgres. We found Foundry's attribute filters did not persist, so isolation by filter was not an option. Per-store isolation makes leakage impossible by construction.

Q54. What does File Search do under the hood?

It chunks uploaded files, embeds the chunks, stores them in a vector store, and at query time retrieves the most relevant chunks - typically with hybrid vector and keyword search and reranking - and feeds them to the model as context. The exact defaults are service-specific, so I say 'typically' and check the docs.

Q55. How do you evaluate a RAG pipeline?

Separate retrieval from generation. Retrieval: recall@k, MRR, nDCG on a labelled set. Generation: faithfulness (is the answer supported by the context), answer relevance, context precision and recall. For extraction specifically: field-level exact-match and normalised-match accuracy against gold documents.

Q56. Why not use RAG for extraction?

Extraction needs the whole document, with fields scattered across it. Top-k retrieval can miss fields or mix documents. So for classification and extraction I attach the single file directly; I still use File Search for submission-wide questions - LOB and chat.

Q57. Why did you add Code Interpreter for spreadsheets?

An SOV is a table. Chunking it destroys row and column structure. Code Interpreter lets the model load the sheet and compute or filter it programmatically, which is far more reliable for tabular data.

Q58. How do you control hallucination in extraction?

Instructions require 'null if not present, do not guess', output restricted to JSON with exactly the given keys, a taxonomy validated in code, low temperature, ACORD handled by a non-generative model, confidence thresholds that route to a human, and underwriter review at Gate 1. Next: schema-enforced outputs and cross-document consistency checks.

Q59. Long context versus RAG?

Long context is simpler and good for a handful of documents, but costs tokens on every call and accuracy can drop for details in the middle. RAG scales to large corpora and is cheaper per query but adds retrieval error. Our submissions are small, so we attach whole documents for extraction and use retrieval only for cross-document questions.

F. Azure AI Foundry and the extraction service

Q60. Take me through process_submission end to end.

Fetch the indexed files. In parallel: one File Search call matches the submission-level LOB against codes from Postgres, and each file is checked for being a broker field list. Then per file, in up to four threads: detect whether it is an ACORD template; if so, send it to the custom Document Intelligence model; otherwise classify against the LOB's document types and extract fields in a chained second call. Write one JSON per document to Blob. Pick the primary document, map the LOB to the canonical string, write a small LOB JSON, and return it all.

Q61. Why classify per document and not per submission?

A submission is a bundle of different documents. LOB is a property of the bundle, but layout and field list are properties of each document. Attaching each file directly also means one document cannot influence another's classification.

Q62. How do you make classification reliable?

The model can only pick from document types stored in Postgres for that LOB. I validate the answer with an exact match, a regex fallback that strips annotation, and a NO_MATCHING_LAYOUT_TYPE outcome rather than forcing a guess. Not-insurance emails return a marker and end the run.

Q63. Why gpt-4o for classify and extract but gpt-5 elsewhere?

On this task gpt-4o was the fastest of the three deployed models and classified the test documents correctly. Chaining only saves time when both calls use the same model, so extraction uses gpt-4o too. LOB match, chat and summary use gpt-5 where the extra reasoning helps.

Q64. What is previous_response_id chaining?

With store=True the Responses API keeps the conversation server-side. A second call that passes previous_response_id continues that conversation, including the file already read, so you do not re-attach it. Saves the dominant cost, reading the file.

Q65. How do you handle rate limits?

One wrapper for every call: catch RateLimitError, sleep 2, 4, 8, 16, 32 seconds, give up after five retries. Plus a concurrency cap of four. At scale I would add jitter and a shared rate limiter across workers.

Q66. Why threads instead of asyncio for the fan-out?

The Foundry SDK calls here are synchronous and I/O-bound. Threads let those waits overlap without rewriting to async clients. The GIL is not a problem for network waits.

Q67. How are ACORD forms handled differently, and why?

A content-based check identifies the template. ACORD is fixed-layout, so a custom Document Intelligence model gives stable field extraction with confidence scores and no generative variance. The LLM only decides 'is this ACORD', not what the fields say.

Q68. What is the broker-supplied field list feature?

Sometimes a submission includes a document that just lists the fields the broker wants. If detected, that list overrides the configured fields for the run, and new field names are written back to the database for that document type so the knowledge base improves over time. The write-back is sequential to avoid races.

Q69. How does configuration-driven extraction work?

A table keyed by (LOB, document_type) holds the extraction_rules - a JSON of field names with descriptions and flags. The classifier's allowed values and the extractor's field list both come from it. Adding a document type or field is a data change.

Q70. What if the model returns invalid JSON?

Strip markdown fences, try json.loads, and if it fails store {raw_response: ...} so nothing is lost and the phase does not crash. To improve: enforce a JSON schema with structured outputs, and retry once with the parse error.

Q71. How would you improve extraction accuracy?

A gold dataset and metrics first, then structured outputs, few-shot examples per document type, field descriptions that disambiguate, validation rules (formats, ranges, cross-field checks), cross-document reconciliation, and a feedback loop from underwriter corrections.

Q72. Why choose a 'primary document'?

Downstream tools - sanctions, perils, risk, Guidewire - need one canonical extraction. The convention is the document type with 'submission' in its name, falling back to ACORD. I would make it an explicit flag.

Q73. You moved from agent versions to the Responses API. Why?

The earlier design created a runtime agent version per run, used it, then deleted it and its context blob. That lifecycle added latency and cleanup risk. Calling the Responses API directly with the file or vector store attached is simpler and lets us chain calls.

Q74. How do you control cost?

Cheap checks first, smaller faster model for per-document work, read each file once via chaining, reuse extracted JSON for the summary, cap tokens, avoid re-indexing, and log token usage per extraction. Next: per-submission cost dashboards.

G. Human-in-the-loop and workflow

Q75. Explain your three gates.

Gate 1 after Phase 7: the underwriter reviews extracted documents and the LOB. Approving triggers Phase 9, licensing and sanctions. Gate 2: the underwriter accepts eligibility, which triggers perils, scoring and the accept/reject recommendation. Gate 3: accepting the risk triggers the Guidewire quote, quote PDF, broker draft and bind-and-issue.

Q76. Why those positions?

Where a human adds the most value per unit of risk. Gate 1 fixes bad input before it contaminates scoring. Gate 2 stops us spending resources on an ineligible risk. Gate 3 sits before irreversible external commitments.

Q77. How are gates implemented technically?

The first segment ends because a router inspects tool results and sends the graph to a halt node when the summary tool succeeds. The later segments end by themselves after their last tool. Resuming is a separate endpoint per gate that starts a fresh narrow graph, with all context in its first message.

Q78. Are the gates defined in the prompt?

No, and that is deliberate. The Phoenix prompt describes the whole sixteen-phase flow and even tells the model to ignore any human-in-the-loop part.

- Gate 1 is enforced by the graph: the main graph is built with a halt switch, a router sees the summary tool succeed and ends the run.
- Gates 2 and 3 are enforced by the structure: each later block is a separate endpoint with its own narrow prompt, so nothing runs until the UI calls it.

Putting approvals in code means a model mistake cannot skip a gate.

Q79. Why not LangGraph's interrupt in production?

Because gates are policy, not the model's decision; humans can take hours; in-memory checkpoints die with a restart; and the UI already owns approval state. interrupt() is the right tool when the approval is part of a single in-flight conversation and you have a durable checkpointer. I prototyped that first.

Q80. What if the underwriter changes extracted data at the gate?

The design principle is that each segment reads artifacts by URL, so a corrected artifact can be used by the next phase. [Confirm how your UI saves edits before you claim this.]

Better design: version the artifact, record who changed what, and pass the version id forward.

Q81. What happens if nobody approves?

Nothing is running, so nothing leaks. The submission stays at its stage in the Workboard. I would add reminders, SLA timers and escalation in the UI or a scheduler.

Q82. How do you prevent double approval or double run?

The UI stage flow is forward-only; the full-pipeline endpoint takes a Postgres lock and returns 409 on a duplicate; phases overwrite their own artifacts; webhooks are idempotent on message id.

Q83. The UI auto-approves above a confidence threshold. Is that safe?

It is a policy choice: high-confidence documents skip manual review, low-confidence ones need a human. To make it safe: calibrate the threshold on a gold set, audit a random sample of auto-approvals, and keep Gates 2 and 3 always human.

Q84. What is your audit trail?

Per-execution step logs in Blob, every artifact in Blob, and stage history and approvals in the Workboard database. To strengthen: link each decision to the prompt version, model deployment and input artifact versions, in an immutable store.

Q85. What other HITL patterns exist?

Approve or reject, edit-then-continue, escalate to a specialist, sampling-based review, and confidence-based routing. Blocking approvals gate irreversible steps; non-blocking audit gates let work continue while a human reviews afterwards.

H. Prompting, LLMOps, evaluation and safety

Q86. Why store prompts in Phoenix?

Non-engineers can review and change them, every change is versioned, and deployments pin a version id, so rollbacks are one line and behaviour is reproducible. It also keeps long prompts out of the code.

Q87. How do you roll out a prompt change safely?

Create a new version, run it on a handful of real submissions in staging, compare outputs against the previous version, then change the pinned id. If it breaks, point the id back. The v101 incident shows why: a malformed version failed immediately and was rolled back by changing one constant.

Q88. What guardrails exist?

A fail-closed master prompt (if anything is missing, unclear or circular, answer End), segment prompts with explicit GUARDRAILS and tool allow-lists written in the prompt, exact-word outputs, JSON-only extraction, a taxonomy validated in code, a not-insurance short-circuit at Phases 1 and 5, recursion limits, and human gates before irreversible steps.

Q89. How do you defend against prompt injection from emails and documents?

Treat all document text as untrusted data. The extraction step only returns JSON for fixed keys; it cannot call tools. The agent's actions are limited by segment, and anything that sends email or binds a policy is behind a gate. Further steps: per-segment tool allow-lists in code, content scanning, and logging suspicious instructions.

Q90. How do you handle PII?

Processing stays inside the Azure tenant and VNet. The weak point is logging full message content to Blob, so I would mask names and addresses in logs, set retention, restrict access and encrypt at rest. Also consider data residency for the model deployment.

Q91. Why different temperatures?

Email metadata extraction uses 0.3 for consistency. The risk justification call uses the default of 1 because it is narrative, but since it is regulator-facing I would lower it. Structured extraction should be as low as the model allows.

Q92. How do you get structured output?

JSON mode on the metadata call and strict prompting plus defensive parsing on the Foundry calls. The stronger approach is schema-enforced structured outputs, with a repair-and-retry step on validation errors.

Q93. How would you evaluate the agent end to end?

Three layers. Component: field-level accuracy for extraction, classification accuracy, scoring unit tests. Trajectory: did the agent call the right tools in the right order with the right arguments. Outcome: final decision matches the underwriter. Use a golden set of anonymised submissions, LLM-as-judge only for narrative parts, run in CI on every prompt change.

Q94. How do you reduce latency and cost?

Fewer LLM-chosen steps, faster model per task, avoid repeated file reads, reuse artifacts, parallelise independent work, backoff to avoid rate-limit stalls, cap tokens. Measure each change with timings like the 111 → 30 s experiment.

Q95. Why did you move rules out of the prompt and into code?

Because LLMs are inconsistent at deterministic transformations. The LOB translation worked once and failed the next run. Putting it in code made it reproducible and testable.

I. Reliability, security and deployment

Q96. What are your main failure modes and mitigations?

Rate limits: backoff and concurrency caps. Long calls through proxies: keep-alives and timeouts. Wrong tool or order: narrow prompts and recursion limits. Label drift: taxonomy validation. Duplicate runs: lock. Bad prompt release: pinned versions. Silent tool omission: a whitelist check. Unparseable output: defensive parsing and fallback.

Q97. A run failed in production. How do you investigate?

Read the AGENT_ERROR record: exception class, last tool, elapsed time, traceback. Open the per-execution log in Blob to see the node and tool sequence. Check container logs. Reproduce the failing tool via REST with the same inputs. Check upstream artifacts in Blob and Postgres. Then fix and re-run only that segment.

Q98. What does your observability look like?

Step-level execution logs in Blob, structured error records, container logs, Phoenix for prompts (and tracing when enabled). I would add per-tool latency and error metrics, token usage per submission and gate wait-time dashboards.

Q99. How is it deployed?

Docker images on Azure Container Instances in a VNet, with internal DNS between agent and tool server, secrets from Key Vault and environment variables, and images pushed from a pipeline. Local development uses virtual environments.

Q100. How are secrets managed?

Environment variables sourced from Key Vault, never in code or prompts. I would move Azure access to managed identity and remove the CLI-credential pattern from the Foundry client.

Q101. How would you make auth between components robust?

Entra ID managed identities with signed tokens, an API gateway in front of the tool server, per-tool scopes, and mTLS or private endpoints inside the VNet.

Q102. What is your testing strategy?

Unit tests for deterministic code: scoring, banding, gate detection, taxonomy validation, mappers. Contract tests between prompts and tools. Recorded-response integration tests. A golden-set evaluation for extraction and a few live end-to-end runs in staging. [Describe what you actually have.]

Q103. How would you handle backups and disaster recovery?

Postgres point-in-time recovery, Blob geo-redundant storage with soft delete, infrastructure as code, and idempotent segments so replaying from the last good artifact is safe.

Q104. What does a new LOB require?

Rows in the taxonomy table (document types and extraction rules), a mapper and payload builders for Guidewire, risk parameters if the scoring differs, and a LOB-to-canonical mapping. The agent code does not change.

J. Insurance domain

Q105. What is a submission, ACORD, SOV, loss run?

A submission is what a broker sends to ask for coverage. ACORD forms are standardised industry application forms. A statement of values lists properties and their insured values. Loss runs are the insured's claims history, usually from prior carriers.

Q106. Quote, bind, issue?

A quote is the offered terms and price. Bind is the commitment to cover. Issue produces the policy documents. Guidewire's bind-and-issue call does the last two together, which is why it sits behind the final gate.

Q107. What is COPE?

Construction, Occupancy, Protection, Exposure - the four classic property-underwriting factors. Our domains A, B, E and C map to them; D covers loss history and financials, and F regulatory and risk management.

Q108. What is an LOB?

Line of business: the type of insurance, such as homeowners, commercial property or management liability.

Q109. What are TIV, PML, flood zone and ISO PPC?

TIV is total insured value. PML is probable maximum loss, here for earthquake as a percentage of TIV. FEMA flood zones classify flood risk. ISO PPC is the fire-protection class rating for a location, based on the local fire department and water supply.

Q110. Why score with bands and not a single accept/reject?

Underwriting is graded. Bands carry an action - auto-accept, accept with conditions, senior review, refer to specialty, decline - so the system routes effort to where human judgement is needed.

Q111. Why screen for sanctions and licensing?

Insurers cannot legally write business with sanctioned parties, and brokers and carriers must be properly licensed in the state. Screening early avoids wasted work and compliance risk.

K. Behavioural and curveballs

Q112. Tell me about a production issue.

Use story 5 (prompt version crash and one-line rollback) or story 2 (LOB translation drift). Structure: what broke, how you found it, what you changed, what you put in place so it cannot recur.

Q113. Tell me about a disagreement.

[Use a true example.] Good shape: two options - e.g. merged classify-and-extract versus chained calls - you proposed an experiment, the data decided, and you reverted your own idea when it lost.

Q114. Why are you looking to move?

Positive and specific: 'I have taken this system from prototype to production and want to work on [larger scale / platform work / a team with mature evaluation and MLOps / deeper agent infrastructure].' Never criticise the current employer.

Q115. What are you proudest of?

Turning extraction from minutes to seconds with measured experiments, and designing the gates so that restarts and human delays cannot lose state.

Q116. Isn't this just prompt engineering?

The prompts are one part. The engineering is the tool layer, the data model, validation, concurrency, rate-limit handling, state and gate design, deterministic scoring, observability and deployment. Prompt changes are versioned and tested like code, and anything with one right answer is moved out of the prompt.

Q117. Did you build this yourself or generate it with AI tools?

Be direct: 'The Workboard UI was generated with Replit because front-end was not our focus. The agent, tool layer and data pipeline are [what I built], and I can walk through any part.' Then follow with a specific design decision to prove it.

Q118. What have you learnt from this project?

Put determinism in code and language tasks in the model; measure before optimising; version prompts like code; design for restarts; and be explicit about trade-offs such as fail-open locks.

L. Core theory, rapid fire

Q119. ReAct pattern?

Reason, Act, Observe in a loop: the model thinks about what to do, calls a tool, reads the observation, and repeats until it can answer. Tool-calling agents implement this with structured calls instead of free-text parsing.

Q120. How does function calling work?

You give the model JSON schemas of functions. Instead of text, it can return a structured call with name and arguments. Your code executes it and returns the result as a tool message; the model then continues. The model never executes anything itself.

Q121. LangChain core concepts?

Runnables and LCEL (compose with the pipe operator, with invoke, batch and stream), prompt templates, chat models, output parsers, retrievers, document loaders and text splitters, and tools. LangGraph is the orchestration layer for stateful, cyclic agents.

Q122. LangChain versus LangGraph?

LangChain gives building blocks. LangGraph gives a runtime for stateful, long-running, branching workflows with persistence, human-in-the-loop and streaming. Prebuilt LangChain agents are built on LangGraph.

Q123. Short-term versus long-term agent memory?

Short-term is the conversation or thread state, persisted by a checkpointer per thread. Long-term is information kept across threads, such as user preferences or learned facts, stored in a key-value or vector store and retrieved when relevant.

Q124. Multi-agent patterns?

Supervisor (a router agent delegates to specialists), handoff or swarm (agents pass control to each other), hierarchical teams, and pipelines. Start with a single agent and tools; add agents only when context or tool sets become unmanageable.

Q125. Agent frameworks compared?

LangGraph: explicit state machine, fine control. AutoGen: conversational multi-agent. CrewAI: role-based crews. Semantic Kernel: Microsoft's SDK with plugins. OpenAI Agents SDK: handoffs, guardrails, tracing. Azure AI Foundry Agent Service: managed agents with built-in tools. Choose by how much control and how much managed infrastructure you want.

Q126. MCP versus A2A?

MCP connects an agent to tools and data. A2A, the Agent2Agent protocol, connects agents to each other. They are complementary.

Q127. RAG versus fine-tuning?

RAG injects fresh, private knowledge at query time and gives traceable sources. Fine-tuning changes behaviour, style or format. They combine. For changing facts, use RAG; for consistent tone or structured output, consider fine-tuning.

Q128. Chunking strategies?

Fixed-size with overlap, recursive character splitting on separators, semantic splitting by meaning, and structure-aware splitting on headings or tables. Trade-off: small chunks retrieve precisely but lose context; large chunks keep context but dilute relevance.

Q129. Embeddings and similarity?

A model maps text to a vector so that similar meanings are close. Cosine similarity compares direction; with normalised vectors it equals dot product. Choose the same model for indexing and querying.

Q130. HNSW versus IVF versus flat indexes?

Flat is exact but linear. IVF clusters vectors and searches the nearest clusters (nprobe) - smaller memory, needs training. HNSW is a navigable graph with high recall and speed at higher memory (tunable M and ef). pgvector supports ivfflat and hnsw.

Q131. Hybrid search and reranking?

Combine keyword (BM25) and vector search, often with reciprocal rank fusion, then rerank the top results with a cross-encoder for precision. Hybrid helps with exact identifiers and rare terms that embeddings blur.

Q132. Metrics for RAG and agents?

RAG: recall@k, MRR, nDCG, faithfulness, answer relevance, context precision and recall. Agents: tool-call accuracy, trajectory correctness, task success rate, steps and cost per task, human override rate.

Q133. Hallucination mitigation?

Grounding in retrieved or attached documents, requiring citations, an explicit 'unknown' option, low temperature, constrained output, programmatic validation, a verification pass, and human review on high-risk outputs.

Q134. Prompt injection types and defences?

Direct (the user writes it) and indirect (hidden in documents, web pages, emails). Defences: least-privilege tools, separate untrusted content from instructions, validate outputs, no automatic privileged actions from model text, human approval for sensitive actions, monitoring.

Q135. JSON mode versus structured outputs?

JSON mode guarantees syntactically valid JSON but not a schema. Structured outputs enforce a JSON schema through constrained decoding. Function calling is another route to structured arguments.

Q136. SSE versus WebSockets?

SSE is one-way server-to-client over HTTP with automatic reconnect, ideal for token and event streaming. WebSockets are bidirectional, for interactive two-way sessions. SSE needs keep-alives behind proxies.

Q137. Asyncio, threads and processes?

Asyncio multiplexes many I/O waits on one thread with await. Threads suit blocking I/O libraries; the GIL limits CPU parallelism but not waiting. Processes parallelise CPU-bound work. Calling a blocking SDK inside an async function stalls the event loop, so wrap it in a thread.

Q138. Why is FastAPI a good fit?

Async by default, pydantic validation, automatic OpenAPI schema, dependency injection, and the OpenAPI schema is what fastapi_mcp turns into MCP tool definitions.

Q139. Prompt caching and semantic caching?

Prompt caching reuses the processed prefix of a long, stable prompt to cut latency and cost. Semantic caching returns a stored answer for a near-duplicate query using embedding similarity; it needs care for correctness.

Q140. What affects LLM cost and latency?

Input and output tokens, model size, retries, number of sequential calls, repeated file reads, and rate-limit stalls. Reduce by routing to smaller models, caching, batching, parallelising independent calls and trimming prompts.

Q141. Temperature and top_p?

Both control randomness. Temperature rescales the distribution; top_p samples from the smallest set of tokens whose probability sums to p. Use low values for extraction and classification, higher for creative text, and change one at a time.

Q142. What is the context window and 'lost in the middle'?

The maximum tokens of input plus output. Models can attend less reliably to information in the middle of very long inputs, so place key instructions at the start or end and prefer targeted context.

Q143. When would you fine-tune a small model for this project?

For high-volume, stable tasks such as document-type classification: a small tuned model can match accuracy at much lower cost and latency. I would collect underwriter-confirmed labels first and keep the large model as a fallback.

Q144. How would you add a new tool safely?

Write the route with a clear operation_id, summary and typed parameters; add it to the whitelist; add a unit test; update the segment prompt and its guardrails; publish a new prompt version; test it; pin the new id.

15. Hands-on drills and cheat sheet

15.1 Coding drills (be able to write these from memory)

Retry with exponential backoff

```
def call_with_retry(fn, max_retries=5, base=2):
    for attempt in range(max_retries + 1):
        try:
            return fn()
        except RateLimitError:
            if attempt == max_retries:
                raise
            time.sleep(base * 2 ** attempt)          # 2, 4, 8, 16, 32  (add random jitter at scale)
```

Parallel fan-out with a concurrency cap

```
with ThreadPoolExecutor(max_workers=min(len(files), 4)) as pool:
    results = list(pool.map(process_file, files))      # preserves input order
```

SSE generator with keep-alive

```
async def generate():
    queue: asyncio.Queue = asyncio.Queue()
    worker = asyncio.create_task(run_graph_into(queue))       # puts ("event", x), ("error", e), ("done", None)
    try:
        while True:
            try:
                kind, payload = await asyncio.wait_for(queue.get(), timeout=20)
            except asyncio.TimeoutError:
                yield ": keepalive\n\n"
                continue
            if kind == "done":
                break
            if kind == "error":
                raise payload
            yield f"data: {payload}\n\n"
    finally:
        worker.cancel()

return StreamingResponse(generate(), media_type="text/event-stream")
```

Minimal tool-calling LangGraph agent

```
from langgraph.graph import StateGraph, START
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import InMemorySaver

class State(TypedDict):
    messages: Annotated[list, add_messages]

llm = model.bind_tools(tools)

def agent(state: State):
    return {"messages": [llm.invoke(state["messages"])]}

g = StateGraph(State)
g.add_node("agent", agent)
g.add_node("tools", ToolNode(tools))
g.add_edge(START, "agent")
g.add_conditional_edges("agent", tools_condition)      # "tools" if tool_calls else END
g.add_edge("tools", "agent")
app = g.compile(checkpointer=InMemorySaver())
app.invoke({"messages": [("user", "...")]}, config={"configurable": {"thread_id": "1"}})
```

Composite score from domain sums

```
def composite(domain_sums, config):          # config[d] = {"max": 250, "weight": 0.25}
    total = 0.0
    for d, cfg in config.items():
        util = max(min(domain_sums[d] / cfg["max"], 1.0), 0.0)
        total += util * cfg["weight"] * 1000
    return round(total, 2)
```

Whiteboard drills

- Architecture: email → agent → MCP → {Foundry, Document Intelligence, Postgres, Blob, Guidewire}; UI ↔ Blob and the three gate endpoints.
- Gates: four segments separated by three gates, with the endpoint under each.
- Extraction: LOB □ field spec, then per-file ACORD or classify-and-extract, then primary document.
- Evolution: Chroma → pgvector → Foundry with the reason and the measured pain at each step.

15.2 Cheat sheet - numbers and names

Item	Value
Phases / gates / tools	16 phases; 3 gates (after Phase 7, 9, 12); 34 MCP operation_ids in about 24 modules
LOB codes	home, commercial, management_liability, newsub, political. Wired to quoting: home → RESIDENTIAL PROPERTY, commercial → COMMERCIAL PROPERTY
Models	Agent: Azure OpenAI chat deployment; Foundry: gpt-5 (LOB, summary, chat), gpt-4o (classify + extract); ACORD: custom Document Intelligence model
Chunking in the Chroma era	800 characters, 100 overlap; embeddings 3072-d in pgvector era
Retry policy	5 retries, delays 2, 4, 8, 16, 32 s; concurrency cap 4
Timeouts	MCP connect 120 s, read 600 s; SSE keep-alive every 20 s
Recursion limits	250 (/chat), 20 (/run/eligibility), 75 (/run/risk, /run/quote-bind), 50 / 75 / 75 (/run/full-pipeline)
Pipeline lock	TTL about 10 minutes; fail-open; HTTP 409; release on failure
Risk	24 parameters, 6 domains A-F, maxima 250/200/200/150/120/80, weights 25/20/20/15/12/8 %; composite 0-1000
Bands	≤200 Auto-Accept; ≤350 Accept; ≤500 Accept with Conditions; ≤650 Senior UW Review; ≤800 Refer to Specialty; ≤1000 Decline
Peril sources	OpenWeather geocoding; USGS earthquake count (10 km, 5 years); nearest-neighbour table
Guidewire	account → submission job (GoCommercialProp) → location (GOCommercialLine) → coverages → quote; later bind-and-issue
Latency results	111 s → 30.5 s chained; 15.8 s gpt-4o chain; SOV 28.8 s, 56 of 56 fields; summary 150+ s → text-only
Prompt versions	v30 Chroma, v34 pgvector, v65 text+PDF layout, v82 Guidewire end-to-end with bind, v97 Foundry, v99 process_submission, v102 fixes v101, v103 current
Ports in code	MCP 5074, agent 5084, HITL prototype 8021, UI 2015 (deployments override)
Run order	MCP server → EmailIntentAgent → UI

15.3 Tables and containers worth knowing by name

Name	Purpose
ref_prompt_library_final_layout	LOB × document_type × extraction_rules JSON; also the classifier's allowed values
submission_index_files	Which files are already uploaded to which per-submission vector store
brt01 ... brt08b	Risk lookup tables: exact match or int4range
risk_weightage / risk_score_band	Domain maxima and weights; score-to-rating bands
PERIL_RISK_DATA	Latitude/longitude keyed peril attributes
guidewire_data	submission_id ↔ Guidewire job number
submissions.full_pipeline_triggered_at	Pipeline lock timestamp
agent-logs / agent-errors (Blob)	Execution logs; structured failure records

15.4 Questions to ask the interviewer

- How do you evaluate your agents and LLM features today - gold sets, LLM-as-judge, online metrics?
- Which human-in-the-loop controls do you require for your highest-risk workflows?
- What is managed versus self-hosted in your AI stack, and how are prompts versioned and rolled back?
- How do you budget and monitor LLM cost per use case?
- What does on-call look like for an AI service, and what are the typical incidents?
- What would a successful first 90 days look like for this role?
- How does the team decide between a workflow, a single agent and a multi-agent design?
- What is the path from engineer to owning a system end to end?

15.5 The night before

- Say the 30-second and 2-minute pitch once each, out loud.
- Redraw the architecture and the gate diagram from memory.
- Re-read Part 13 and promise yourself not to over-claim.
- Pick your three STAR stories and the two gaps you will volunteer.
- Sleep. Fluency beats cramming.